Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial
Semester I Tahun 2026/2027



Disusun oleh:

Wildan Abdurrahman Ghazali	13524054
Nathanael Gunawan	13524066
Dika Pramudya Nugraha	13524132

INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2026

DAFTAR ISI

BAB 1
Deskripsi Persoalan 2
BAB 2
Pembahasan 3
2.1 Representasi Data dan State 3
2.2 Fungsi Objektif dan Validasi 4
2.2.1 Fungsi Objektif Utama 4
2.2.2 Fungsi Objektif Alternatif 5
2.2.3 Validasi Geometri dan Kendala 6
2.3 Pembangkitan State Awal dan Neighbor 8
2.3.1 Pembangkitan State Awal 8
2.3.2 Operator Perubahan State 9
2.3.3 Pemilihan Neighbor secara Acak 10
2.4 Steepest Ascent Hill-Climbing 12
2.5 Variasi Hill-Climbing 14
2.5.1 Hill-Climbing with Sideways Move 15
2.5.2 Random Restart Hill-Climbing 16
2.5.3 Stochastic Hill-Climbing 16
2.6 Simulated Annealing 17
2.7 Genetic Algorithm 23
2.8 Pengembangan Multi-Truk 28
2.9 Menu, penyimpanan, dan visualisasi 29
BAB 3
Eksperimen dan Analisis 31
3.1 Konfigurasi Eksperimen 31
3.1.1 Dataset dan Skenario Pengujian 31
3.1.2 Konfigurasi Hill-Climbing 32
3.1.3 Konfigurasi Simulated Annealing 33
3.1.4 Konfigurasi Genetic Algorithm 33
3.1.5 Pencatatan dan Penyimpanan Hasil 34
3.2 Hasil pencarian dan kedekatan terhadap optimum 35
3.3 Probabilitas penerimaan dan optimum lokal SA 47
3.4 Pengaruh parameter Genetic Algorithm 48
3.5 Perbandingan durasi, konsistensi, dan verifikasi 50
BAB 4 51

Kesimpulan dan Saran **51**
4.1 Kesimpulan 51
4.2 Saran 51
Pembagian Tugas **52**
Referensi **53**
Lampiran **54**

BAB 1

Deskripsi Persoalan

Permasalahan yang diselesaikan adalah optimasi pemilihan dan penyusunan paket dalam peti kemas truk agar total nilai (*value*) paket yang diangkut maksimum. Setiap truk memiliki batas dimensi dan kapasitas berat. Paket berbentuk kubus atau balok dengan atribut dimensi, nilai, berat, status mudah pecah (*fragile*), dan estimasi waktu kedatangan (ETA). Tidak semua paket harus diangkut, paket yang tidak dipilih tetap berada di luar truk [2].

Solusi direpresentasikan sebagai pemetaan setiap ID paket ke posisi sudut kiri, bawah, belakang (x, y, z), orientasi, dan indeks truk yang mengangkutnya. Koordinat berupa bilangan bulat. Orientasi menentukan pemetaan sisi lebar, panjang, dan tinggi paket terhadap sumbu koordinat, dengan enam kemungkinan permutasi. Paket yang tidak diangkut memiliki indeks truk *None*. Setiap truk menggunakan ruang koordinat lokalnya sendiri, sedangkan paket di luar direpresentasikan dalam ruang terpisah.

Susunan paket harus memenuhi sejumlah kendala. Seluruh bagian paket yang dimuat harus berada di dalam batas peti kemas, tidak saling beririsan dengan paket lain dalam truk yang sama, dan tidak menyebabkan total berat melebihi kapasitas truk. Paket di lantai dianggap memiliki penyangga. Paket di atas lantai harus memiliki setidaknya satu titik alas yang ditopang permukaan atas paket lain dalam truk yang sama. Paket *fragile* tidak boleh menjadi penyangga paket lain [2].

Fungsi objektif utama memaksimalkan jumlah nilai seluruh paket yang diangkut. Pada pengembangan multi-truk, jumlah tersebut dihitung untuk seluruh armada, dengan jumlah truk ditentukan melalui input. Implementasi juga menyediakan fungsi objektif alternatif yang mempertimbangkan urgensi ETA menggunakan bobot $1/(1 + ETA)$, tanpa menambahkan atribut baru.

Perubahan kandidat solusi mencakup pertukaran penempatan dua paket, perpindahan paket, dan rotasi 90 derajat terhadap salah satu sumbu. Pada kasus multi-truk, perpindahan dan pertukaran dapat melibatkan truk yang berbeda. Kandidat yang digunakan dalam pencarian harus memenuhi seluruh kendala penempatan.

BAB 2

Pembahasan

2.1 Representasi Data dan State

Program diimplementasikan menggunakan Python dengan pemisahan antara model data, validasi geometri, pembangkitan kandidat solusi, algoritma pencarian, serta input dan output. Pemisahan ini memungkinkan algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm menggunakan representasi persoalan dan pemeriksaan kendala yang sama.

Model data didefinisikan pada src/core/models.py menggunakan dataclass. Komponen utama model ditunjukkan pada Tabel 2.1.

Tabel 2.1 Komponen model data

Kelas	Fungsi
Dimensions	Menyimpan lebar, panjang, dan tinggi yang berupa bilangan bulat positif.
Truck	Menyimpan dimensi peti kemas dan kapasitas berat maksimum.
Package	Menyimpan ID, dimensi, nilai, berat, status fragile, dan ETA paket.
Problem	Menyimpan kumpulan truk dan paket yang digunakan dalam pencarian.
Placement	Menyimpan posisi, orientasi, dan indeks truk suatu paket.
State	Memetakan setiap ID paket ke objek Placement.
SearchResult	Menyimpan konfigurasi, state awal dan akhir, nilai objektif, durasi, iterasi, riwayat, serta metadata pencarian.

Problem menggunakan atribut trucks berupa tuple sehingga dapat merepresentasikan satu maupun beberapa truk. Jumlah truk harus sedikitnya satu, sedangkan setiap paket harus memiliki ID yang unik.

Posisi paket dinyatakan sebagai koordinat sudut kiri-bawah-belakang (x,y,z). Orientasi menyatakan pemetaan sisi lebar, panjang, dan tinggi paket terhadap sumbu koordinat. Enam kemungkinan orientasi diperoleh dari permutasi sumbu x, y, dan z.

Atribut penempatan paket didefinisikan sebagai berikut:

```
@dataclass(frozen=True)
class Placement:
    position: tuple[int, int, int]
    orientation: tuple[str, str, str] = ("x", "y", "z")
    truck_index: int | None = None
```

Nilai *truck_index* menunjukkan indeks truk yang mengangkut paket. Nilai *None* menunjukkan bahwa paket berada di luar seluruh truk. Status *inside* dihitung melalui properti berikut sehingga tidak perlu disimpan sebagai atribut yang dapat berubah secara terpisah.

Berikut merupakan cuplikan kode status paket berdasarkan indeks truk:

```
@property
def inside(self) -> bool:
    return self.truck_index is not None
```

Setiap truk memiliki ruang koordinat lokal. Oleh karena itu, dua paket pada truk berbeda dapat memiliki koordinat yang sama tanpa saling beririsan. Paket yang tidak diangkut direpresentasikan dalam ruang luar yang terpisah dari ruang setiap truk.

Objek *Placement* bersifat *immutable*. Pembentukan kandidat dilakukan dengan menyalin *dictionary* penempatan pada *state* dan mengganti objek penempatan yang diperlukan. Pendekatan ini menjaga agar pembentukan kandidat tidak mengubah *state* asal.

2.2 Fungsi Objektif dan Validasi

2.2.1 Fungsi Objektif Utama

Fungsi objektif utama memaksimalkan total nilai paket yang diangkut [2]. Misalkan P merupakan himpunan paket dan $I_i(s)$ bernilai satu apabila paket i berada di dalam salah satu truk pada state s, serta nol apabila berada di luar. Fungsi objektif dinyatakan sebagai:

$$f_{value}(s) = \sum_{i \in P} v_i I_i(s)$$

dengan v_i merupakan nilai paket i.

Implementasi fungsi tersebut terdapat pada src/core/geometry.py.

Cuplikan kode evaluasi fungsi objektif utama sebagai berikut:

```
def objective(problem: Problem, state: State) -> float:
    errors = validation_errors(problem, state)
    if errors:
        raise ValueError("Invalid state: " + "; ".join(errors))
    return sum(p.value for p in problem.packages if
state.placements[p.id].inside)
```

State harus lolos validasi sebelum memperoleh nilai objektif. Dengan demikian, pelanggaran kendala tidak ditangani menggunakan penalti, melainkan dengan menolak state yang tidak valid.

2.2.2 Fungsi Objektif Alternatif

Sebagai pengembangan bonus, program menyediakan fungsi objektif *eta_priority*. Fungsi ini menggunakan atribut nilai dan ETA yang sudah tersedia, dimana nilai paket yang sama, ETA yang lebih kecil memberikan kontribusi lebih besar. Penambahan satu pada penyebut menghindari pembagian dengan nol ketika ETA bernilai nol. Fungsi objektif dinyatakan sebagai :

$$f_{value}(s) = \sum_{i \in P} \frac{v_i I_i(s)}{ETA_i + 1}$$

Cuplikan kode evaluasi fungsi objektif alternatif sebagai berikut:

```
def objective_eta_priority(problem: Problem, state: State) ->
float:
    errors = validation_errors(problem, state)
    if errors:
        raise ValueError("Invalid state: " + "; ".join(errors))
    return round(sum(p.value / (1 + p.eta) for p in
problem.packages if state.placements[p.id].inside), 6)
```

Fungsi *objective_eta_priority()* memvalidasi *state*, menghitung jumlah nilai berbobot, kemudian membulatkan hasil hingga enam angka desimal. Pemilihan fungsi objektif dilakukan melalui *get_objective()* berdasarkan nama *value* atau *eta_priority*.

Fungsi alternatif ini mengubah prioritas pemilihan muatan. ETA tidak digunakan untuk menghitung rute atau memastikan tenggat pengiriman terpenuhi.

2.2.3 Validasi Geometri dan Kendala

Fungsi *oriented_dimensions()* menghitung ukuran paket sepanjang sumbu koordinat setelah orientasi diterapkan. Fungsi *bounds()* menghasilkan koordinat batas bawah dan batas atas paket, sedangkan *overlap()* memeriksa irisan dua balok.

Cuplikan kode pemeriksaan irisan balok sebagai berikut:

```
def overlap(a: tuple, b: tuple,
            axes: tuple[int, ...] = (0, 1, 2)) -> bool:
    return all(max(a[0][i], b[0][i]) < min(a[1][i], b[1][i])
               for i in axes)
```

Perbandingan menggunakan operator < (kurang dari), sehingga dua paket yang hanya bersentuhan pada permukaannya tidak dianggap saling beririsan.

Fungsi *validation_errors()* memeriksa kendala berikut:

1. State memuat setiap ID paket tepat satu kali.
2. Setiap penempatan merupakan objek Placement.
3. Indeks truk, apabila digunakan, menunjuk truk yang tersedia.
4. Seluruh bagian paket yang dimuat berada di dalam batas peti kemas.
5. Paket dalam truk yang sama tidak saling beririsan.
6. Total berat paket pada setiap truk tidak melebihi kapasitasnya.
7. Paket di atas lantai memiliki penyangga.
8. Paket fragile tidak menjadi penyangga paket lain.

Berikut cuplikan kode validation_errors:

```
def validation_errors(problem: Problem, state: State) ->
list[str]:
    packages = problem.package_by_id
    if set(state.placements) != set(packages):
        return ["State must contain each package id exactly
once"]
    if any(not isinstance(p, Placement) for p in
state.placements.values()):
        return ["State entries must be Placement objects"]
    errors = []
    for key, placement in state.placements.items():
        index = placement.truck_index
        if index is not None:
```

```
            if type(index) is not int or not 0 <= index <
len(problem.trucks):
                errors.append(f"{key}: invalid truck index")
    if errors:
        return errors
    boxes = {key: bounds(packages[key], p) for key, p in
state.placements.items()}
    for key, p in state.placements.items():
        low, high = boxes[key]
        if p.inside:
            truck = problem.trucks[p.truck_index].dimensions
            truck_bounds = ((0, 0, 0), (truck.w, truck.l,
truck.h))
            if not all(0 <= low[i] and high[i] <=
truck_bounds[1][i] for i in range(3)):
                errors.append(f"{key}: inside package crosses
truck boundary")
        elif any(overlap(boxes[key], ((0, 0, 0),
(truck.dimensions.w, truck.dimensions.l, truck.dimensions.h)))
                 for truck in problem.trucks):
            errors.append(f"{key}: outside package must be
completely outside every truck")
    for key, other in combinations(packages, 2):
        idx_a, idx_b = state.placements[key].truck_index,
state.placements[other].truck_index
        if idx_a == idx_b and overlap(boxes[key],
boxes[other]):
            errors.append(f"{key}/{other}: packages overlap")
    for index, truck in enumerate(problem.trucks):
        inside = [key for key, p in state.placements.items() if
p.truck_index == index]
        if sum(packages[key].weight for key in inside) >
truck.max_capacity:
            errors.append(f"Truck {index}: weight capacity
exceeded")
        for key in inside:
            if boxes[key][0][2] == 0:
                continue
            supports = [other for other in inside if other !=
key
                        and boxes[other][1][2] ==
boxes[key][0][2]
                        and overlap(boxes[key], boxes[other],
(0, 1))]
```

```
            if not supports:
                errors.append(f"{key}: no base point has
support")
            for other in supports:
                if packages[other].is_fragile:
                    errors.append(f"{other}: fragile package
supports {key}")
    return errors
```

Paket dengan koordinat dasar $z = 0$ dianggap memperoleh penyangga dari lantai. Untuk paket di atas lantai, penyangga harus mempunyai permukaan atas tepat pada ketinggian dasar paket dan beririsan pada proyeksi bidang x, y. Penyangga dicari hanya dari paket dalam truk yang sama.

Pemeriksaan mengikuti ketentuan bahwa setidaknya satu titik alas memperoleh support; implementasi tidak mensyaratkan seluruh luas alas tertopang. Pada representasi yang digunakan, overlap antar paket di ruang luar tidak diperiksa oleh validator [2].

2.3 Pembangkitan State Awal dan Neighbor

2.3.1 Pembangkitan State Awal

Pembangkitan state awal diimplementasikan melalui *random_state()* pada *src/core/search_space.py*. Proses dimulai dari *empty_state()*, yaitu kondisi ketika seluruh paket berada di luar truk. Fungsi *outside_position()* menyediakan posisi parkir dengan jarak berdasarkan dimensi maksimum paket.

Selanjutnya, urutan paket diacak. Untuk setiap paket, program memilih orientasi awal dan memiliki peluang 30% untuk melewati upaya penempatannya. Jika penempatan dicoba, program mengacak urutan orientasi, urutan truk, dan koordinat dalam truk tujuan.

Bagian pemilihan truk ditunjukkan pada cuplikan kode berikut:

```
truck_indices = list(range(len(problem.trucks)))
rng.shuffle(truck_indices)
for truck_index in truck_indices:
    positions = list(inside_positions(problem, truck_index))
    rng.shuffle(positions)
    state = _try_placements(
        problem, state, package.id,
        orientations, positions, truck_index
```

```
    )
    if state.placements[package.id].inside:
        break
```

Fungsi *_try_placements()* menerima kandidat pertama yang lolos *is_valid()*. Jika paket tidak dapat ditempatkan pada seluruh truk yang dicoba, paket tetap berada di luar.

Peluang pelewatan 30% merupakan heuristik implementasi untuk memvariasikan muatan awal. Angka tersebut bukan jaminan bahwa 70% paket akan dimuat karena penempatan juga dapat gagal akibat kendala. Proses ini menghasilkan state awal secara acak, tetapi tidak mengambil seluruh kemungkinan state valid dengan probabilitas yang seragam.

2.3.2 Operator Perubahan State

Sesuai spesifikasi tugas, program menyediakan tiga operator pembentuk kandidat, yaitu pertukaran dua paket, perpindahan satu paket, dan rotasi 90 derajat terhadap salah satu sumbu [2].

Tabel 2.2 Operator perubahan *state*

Operator	Perubahan
move_package()	Mengubah posisi dan truk tujuan satu paket. Nilai truk None memindahkan paket ke luar.
swap_packages()	Menukar posisi dan indeks truk dua paket, dengan mempertahankan orientasi masing-masing.
rotate_package()	Mengubah orientasi melalui rotasi 90 derajat terhadap sumbu yang dipilih, tanpa mengubah posisi dan truk.

Operator dapat menghasilkan kandidat tidak valid. Pemeriksaan kelayakan dilakukan oleh fungsi pembangkit neighbor sebelum kandidat digunakan oleh algoritma.

Fungsi *neighbors()* membangkitkan kandidat pertukaran, perpindahan, dan rotasi. Perpindahan di dalam mencakup seluruh koordinat integer pada setiap truk. Perpindahan ke luar dibatasi pada slot parkir yang disediakan agar enumerasi tetap berhingga. Kandidat yang sama dengan state asal, duplikat, atau tidak valid tidak dikembalikan.

2.3.3 Pemilihan Neighbor secara Acak

```
def neighbors(problem: Problem, state: State) ->
Iterator[State]:
    if not is_valid(problem, state):
        raise ValueError("Neighborhood requires a valid source
state")
    ids = [p.id for p in problem.packages]
    parking_positions = [outside_position(problem, slot) for
slot in range(len(ids))]

    def raw_candidates() -> Iterator[State]:
        for a, b in combinations(ids, 2):
            yield swap_packages(state, a, b)
        for key in ids:
            for truck_index in range(len(problem.trucks)):
                for position in inside_positions(problem,
truck_index):
                    yield move_package(state, key, position,
truck_index)
            for position in parking_positions:
                yield move_package(state, key, position, None)
            for axis in ("x", "y", "z"):
                yield rotate_package(state, key, axis)

    def signature(s: State) -> tuple:
        return tuple(s.placements[key] for key in ids)

    seen = {signature(state)}
    for candidate in raw_candidates():
        key = signature(candidate)
        if key not in seen and is_valid(problem, candidate):
            seen.add(key)
            yield candidate
```

Ada tiga cara sampling pada modul *search_space*. *sample_neighbor* memakai *reservoir sampling* atas seluruh *iterator neighbors* dan tersedia sebagai *helper*. SA dan mutasi GA sekarang memakai *random_neighbor* dengan maksimum 100 percobaan move acak. Stochastic HC memakai *random_successor* yang mengacak indeks *move* tanpa pengulangan sampai memperoleh kandidat valid atau semua move habis. Ketiga fungsi tersebut mempunyai biaya dan distribusi proposal yang berbeda.

```
def _try_placements(problem: Problem, state: State, package_id:
str, orientations, positions) -> State:
    for orientation in orientations:
        for position in positions:
            candidate = state.copy()
            candidate.placements[package_id] =
Placement(position, orientation, True)
            if is_valid(problem, candidate):
                return candidate
    return state

def random_neighbor(problem: Problem, state: State, rng:
Random, tries: int = 100) -> State | None:
    if not is_valid(problem, state):
        raise ValueError("Neighborhood requires a valid source
state")
    if not problem.packages:
        return None
    ids = [p.id for p in problem.packages]
    n = len(ids)
    available_moves = ["move", "rotate", "swap"] if n >= 2 else
["move", "rotate"]

    for _ in range(tries):
        move_type = rng.choice(available_moves)
        if move_type == "swap":
            a, b = rng.sample(ids, 2)
            candidate = swap_packages(state, a, b)
        elif move_type == "rotate":
            pkg_id = rng.choice(ids)
            axis = rng.choice(("x", "y", "z"))
            candidate = rotate_package(state, pkg_id, axis)
        else:  # move
            pkg_id = rng.choice(ids)
            if rng.random() < 0.5:
                truck_index =
rng.randrange(len(problem.trucks))
                d = problem.trucks[truck_index].dimensions
                # Coba tempatkan di dalam truk
                inside_tops = [
                    state.placements[other.id].position[2] +
oriented_dimensions(other, state.placements[other.id])[2]
                    for other in problem.packages
```

```
                    if other.id != pkg_id and
state.placements[other.id].truck_index == truck_index
                ]
                possible_z = [0] + [z for z in inside_tops if z
< d.h]
                z = rng.choice(possible_z)
                x = rng.randrange(d.w)
                y = rng.randrange(d.l)
                candidate = move_package(state, pkg_id, (x, y,
z), truck_index)
            else:
                # Pindahkan ke slot parkir luar
                slot = rng.randrange(n)
                candidate = move_package(state, pkg_id,
outside_position(problem, slot), None)

        if candidate.placements != state.placements and
is_valid(problem, candidate):
            return candidate

    return None
```

2.4 Steepest Ascent Hill-Climbing

Steepest Ascent Hill-Climbing memilih neighbor dengan nilai evaluasi tertinggi dan menerima perpindahan apabila nilai tersebut lebih baik daripada state saat ini. Pencarian berhenti ketika tidak tersedia neighbor yang memberikan peningkatan [1].

Algoritma Steepest Ascent Hill-Climbing diimplementasikan melalui fungsi `solve()` pada src/algorithms/`hill_climbing.py`. Fungsi ini menerima problem, konfigurasi, dan seed. Konfigurasi objective menentukan evaluator melalui `get_objective()`, dengan value sebagai pilihan bawaan dan `eta_priority` sebagai pilihan alternatif.

Fungsi `_best_neighbor()` memindai seluruh neighbor valid dan memilih kandidat dengan nilai objektif tertinggi. Jika beberapa kandidat mempunyai nilai tertinggi yang sama, reservoir sampling memilih salah satunya menggunakan generator bilangan acak lokal. Fungsi `solve()` hanya menerima perpindahan yang meningkatkan nilai objektif. Pencarian berhenti ketika tidak ada neighbor yang lebih baik atau batas `max_iterations` tercapai. Nilai None pada batas tersebut berarti pencarian berjalan tanpa batas iterasi.

Cuplikan kode pemilihan neighbor dan pencarian sebagai berikut:

```
def _best_neighbor(problem: Problem, current: State, value:
float, rng: Random,
                   allow_equal: bool = False, score=objective)
-> tuple[State | None, float, int]:
    best, best_value = None, value
    tied = evaluated = 0
    for candidate in neighbors(problem, current):
        evaluated += 1
        candidate_value = score(problem, candidate)
        if candidate_value > best_value:
            best, best_value, tied = candidate,
candidate_value, 1
        elif candidate_value == best_value and (best is not
None or allow_equal):
            tied += 1
            if rng.randrange(tied) == 0:
                best = candidate
    return best, best_value, evaluated

def solve(problem: Problem, config: dict | None = None, seed:
int | None = None) -> SearchResult:
    config = dict(config or {})
    unknown = set(config) - {"initial_state", "max_iterations",
"objective"}
    if unknown:
        raise ValueError(f"Unknown HC parameters:
{sorted(unknown)}")
    objective_name = config.get("objective", "value")
    score = get_objective(objective_name)
    limit = config.get("max_iterations")
    if limit is not None:
        integer(limit, "max_iterations", 0)
    started = perf_counter()
    rng = Random(seed)
    initial = config.get("initial_state")
    if initial is None:
        initial = random_state(problem, seed)
    if not isinstance(initial, State) or not is_valid(problem,
initial):
        raise ValueError("initial_state must be a valid State")
    initial = initial.copy()
    current = initial.copy()
    value = score(problem, current)
```

```
    history = [{"iteration": 0, "objective": value, "accepted":
False, "neighbors_evaluated": 0}]
    scans = 0
    accepted_moves = 0
    reason = "max_iterations"
    while limit is None or scans < limit:
        best, best_value, evaluated = _best_neighbor(problem,
current, value, rng, score=score)
        scans += 1
        accepted = best_value > value
        history.append({"iteration": scans, "objective":
best_value, "accepted": accepted,
                        "neighbors_evaluated": evaluated})
        if not accepted:
            reason = "no_improving_neighbor"
            break
        current, value = best, best_value
        accepted_moves += 1
    return SearchResult("hc", {"variant": "steepest_ascent",
"max_iterations": limit, "objective": objective_name},
                        seed, initial, current.copy(), value,
perf_counter() - started, scans, history,
                        stop_reason=reason,
metadata={"accepted_moves": accepted_moves})
```

State awal diperoleh dari `random_state()` atau parameter `initial_state` yang telah lolos validasi, kemudian disalin sebelum pencarian. Setiap iterasi menghitung satu pemindaian neighborhood, termasuk pemindaian terakhir yang tidak menghasilkan perpindahan. Jumlah perpindahan yang diterima dicatat pada `metadata.accepted_moves`.

Karena perpindahan bernilai sama ditolak, perubahan posisi, orientasi, atau truk yang belum meningkatkan nilai objektif tidak diterima. Pencarian dapat berhenti pada susunan yang masih memerlukan perubahan setara sebelum paket lain dapat dimasukkan.

2.5 Variasi Hill-Climbing

Ketiga variasi Hill-Climbing diimplementasikan pada src/algorithms/`hill_climbing_variants.py` melalui fungsi `solve_sideways()`, `solve_random_restart()`, dan `solve_stochastic()`. Seluruhnya menggunakan model state, validasi, dan pilihan fungsi objektif yang sama.

2.5.1 Hill-Climbing with Sideways Move

Hill-Climbing with Sideways Move memungkinkan perpindahan ke neighbor dengan nilai evaluasi yang sama untuk menelusuri bagian ruang pencarian yang datar. Materi kuliah membatasi jumlah perpindahan setara secara berturut-turut [1].

Fungsi `solve_sideways()` menjalankan pencarian melalui `_climb()`. Pemilihan neighbor memakai `_best_neighbor()` dengan `allow_equal`=True, sehingga kandidat bernilai sama dapat dipilih apabila tidak tersedia peningkatan. Kandidat yang meningkatkan nilai objektif tetap diprioritaskan.

Parameter `max_sideways_moves` membatasi total perpindahan setara yang diterima selama satu pencarian, dengan nilai bawaan 100. Hitungan total tidak direset setelah peningkatan. Program juga mencatat `consecutive_sideways_moves`, yang direset setelah peningkatan, tetapi hitungan beruntun tersebut tidak menjadi batas penghentian.

Pencarian berhenti segera setelah perpindahan setara mencapai batas, ketika tidak ada kandidat yang dapat diterima, atau ketika batas iterasi tercapai. Nilai `max_sideways_moves` sebesar nol menonaktifkan perpindahan setara. Bagian pembaruan hitungan pada `_climb()` sebagai berikut:

```
        chosen, chosen_value, evaluated =
hill_climbing._best_neighbor(
            problem, current, value, rng, allow_equal=True,
score=score)
        scans += 1
        accepted = chosen is not None and (chosen_value > value
or sideways_moves < sideways_limit)
        is_sideways = accepted and chosen_value == value
        if accepted:
            current, value = chosen, chosen_value
            accepted_moves += 1
            sideways_moves += int(is_sideways)
            consecutive_sideways_moves =
consecutive_sideways_moves + 1 if is_sideways else 0
        record(accepted, evaluated, is_sideways)
        if not accepted:
            reason = "max_sideways_moves" if chosen is not None
else "no_improving_neighbor"
            break
        if is_sideways and sideways_moves == sideways_limit:
            reason = "max_sideways_moves"
            break
```

2.5.2 Random Restart Hill-Climbing

Random Restart Hill-Climbing menjalankan pencarian berulang dari state awal acak untuk mengurangi ketergantungan hasil terhadap satu titik awal [1].

Fungsi `solve_random_restart()` menjalankan Steepest Ascent Hill-Climbing dari beberapa state awal. Parameter `max_restarts` mempunyai nilai bawaan 10. Jika nilainya R, program menjalankan satu pencarian awal dan R pencarian ulang, sehingga jumlah pencarian adalah R + 1.

Parameter `initial_state`, jika diberikan, hanya digunakan pada pencarian pertama. Pencarian berikutnya membangkitkan state acak dengan seed turunan dari rng.getrandbits(64). Batas `max_iterations` berlaku untuk setiap pencarian, sedangkan iterations pada hasil akhir merupakan jumlah iterasi seluruh pencarian.

State akhir diambil dari pencarian dengan nilai objektif tertinggi. Riwayat menyimpan indeks restart, iterasi dalam pencarian, iterasi gabungan, dan `best_objective`. Metadata memuat `restart_count`, `iterations_per_restart`, `best_restart`, serta state, seed, durasi, dan alasan berhenti setiap pencarian. Bagian pembangkitan state tiap pencarian sebagai berikut:

```
    for restart in range(max_restarts + 1):
        run_seed = seed if restart == 0 else
rng.getrandbits(64)
        run_config = {"max_iterations": limit, "objective":
config["objective"]}
        if restart == 0:
            run_config["initial_state"] =
config.get("initial_state")
        else:
            run_config["initial_state"] = random_state(problem,
run_seed)
        result = hill_climbing.solve(problem, run_config,
run_seed)
```

2.5.3 Stochastic Hill-Climbing

Stochastic Hill-Climbing membangkitkan successor secara acak dan menerima perpindahan apabila kandidat tersebut memiliki nilai yang lebih baik daripada state saat ini [1].

Fungsi `solve_stochastic()` mengambil satu kandidat melalui `random_successor()` pada setiap iterasi. Pada src/core/`search_space.py`, fungsi tersebut meneruskan pemanggilan ke `random_neighbor()` dengan batas bawaan 100 percobaan. Pemilihan ini tidak memindai seluruh neighborhood dan tidak menjamin distribusi neighbor yang seragam.

Kandidat hanya diterima apabila nilai objektifnya lebih tinggi. Kandidat setara atau lebih buruk ditolak, lalu pencarian mencoba lagi pada iterasi berikutnya. Jika sampling mengembalikan None, iterasi tetap dicatat dan pencarian berhenti dengan `sampling_attempts_exhausted`. Kondisi ini tidak membuktikan bahwa seluruh neighborhood tidak memiliki kandidat valid.

Batas `max_iterations` harus berhingga untuk membatasi percobaan setelah penolakan. Jika tidak diberikan atau bernilai None, solver menggunakan 1000 iterasi. Cuplikan kode penerimaan kandidat sebagai berikut:

```
        candidate = random_successor(problem, current, rng)
        if candidate is None:
            history.append({"iteration": iterations,
"objective": value, "accepted": False, "neighbors_evaluated":
0})
            reason = "sampling_attempts_exhausted"
            break
        candidate_value = score(problem, candidate)
        accepted = candidate_value > value
        if accepted:
            current, value = candidate, candidate_value
            accepted_moves += 1
        history.append({"iteration": iterations, "objective":
value, "accepted": accepted, "neighbors_evaluated": 1})
```

Riwayat Stochastic Hill-Climbing menyimpan nilai objektif state yang dipertahankan, keputusan penerimaan, dan `neighbors_evaluated`. Nilai `neighbors_evaluated` sebesar satu menunjukkan satu kandidat valid telah dievaluasi, sedangkan nol menunjukkan sampling gagal; angka tersebut tidak menghitung seluruh percobaan pembentukan kandidat. Jumlah perpindahan yang diterima disimpan pada `metadata.accepted_moves`.

2.6 Simulated Annealing

Simulated Annealing memungkinkan perpindahan ke kandidat yang lebih buruk dengan probabilitas yang bergantung pada perubahan nilai objektif dan temperatur. Untuk

$\Delta E = f(s') - f(s) < 0$, probabilitas penerimaannya adalah $\exp(\Delta E/T)$. Penurunan temperatur mengurangi peluang penerimaan penurunan nilai dengan besar yang sama [1].

Simulated Annealing diimplementasikan melalui fungsi `solve()` pada src/algorithms/`simulated_annealing.py`. Konfigurasi memuat `initial_state`, `initial_temperature`, `max_iterations`, dan objective. Nilai bawaan suhu awal adalah 100 dan batas iterasi adalah 1000. Suhu awal harus positif, sedangkan batas iterasi merupakan bilangan bulat minimal satu.

Program menggunakan jadwal linear T(t) = T0 × max(0, 1 − (t − 1)/N), dengan T0 sebagai suhu awal, N sebagai `max_iterations`, dan t dimulai dari satu. Suhu pada percobaan pertama adalah T0 dan pada percobaan ke-N adalah T0/N. Pemeriksaan berikutnya menghasilkan suhu nol, sehingga pencarian berhenti dengan `temperature_zero`.

Setiap iterasi memanggil `random_neighbor()` untuk mengambil satu kandidat valid yang berbeda dari state saat ini. Fungsi tersebut mencoba perpindahan, rotasi, atau pertukaran secara acak dengan maksimum 100 percobaan. Perpindahan ke dalam memilih truk tujuan, mengacak x dan y, serta memilih z dari lantai atau permukaan atas paket pada truk tujuan. Jika seluruh percobaan gagal, fungsi mengembalikan None. Kegagalan ini menghabiskan satu iterasi SA, mempertahankan state serta status stuck, dan dicatat melalui `candidate_found`=False.

Selisih nilai objektif dihitung sebagai ΔE = f(kandidat) − f(current). Kandidat dengan ΔE ≥ 0 selalu diterima. Kandidat dengan ΔE < 0 diterima apabila bilangan acak lebih kecil dari exp(ΔE/T), sehingga probabilitas penerimaan adalah p = min(1, exp(ΔE/T)).

Fungsi `_exponential_metrics()` mencatat nilai eksponensial mentah pada `exp_delta_over_temperature`, termasuk nilai di atas satu. Probabilitas yang digunakan untuk keputusan penerimaan disimpan terpisah pada `acceptance_probability`. Jika eksponensial melampaui jangkauan float, nilai mentah disimpan sebagai None dan `exp_overflow`=True. Nilai ΔE/T tetap dicatat pada `log_exp_delta_over_temperature`, menggunakan teks hasil perhitungan Decimal jika rasionya tidak berhingga sebagai float.

Cuplikan kode fungsi `solve()` sebagai berikut:

```
def solve(problem: Problem, config: dict | None = None, seed:
int | None = None) -> SearchResult:
    config = dict(config or {})

    unknown = set(config) - {"initial_state",
"initial_temperature", "max_iterations", "objective"}
```

```
    if unknown:
        raise ValueError(f"Unknown SA parameters:
{sorted(unknown)}")

    objective_name = config.get("objective", "value")
    score = get_objective(objective_name)
    initial_temperature = config.get("initial_temperature",
100.0)
    max_iterations = config.get("max_iterations", 1000)

    number(initial_temperature, "initial_temperature")
    if initial_temperature == 0:
        raise ValueError("initial_temperature must be greater
than 0")

    integer(max_iterations, "max_iterations", 1)

    rng = Random(seed)
    started = perf_counter()

    initial = config.get("initial_state")
    if initial is None:
        initial = random_state(problem, seed)

    if not isinstance(initial, State) or not is_valid(problem,
initial):
        raise ValueError("initial_state must be a valid State")

    initial = initial.copy()
    current = initial.copy()
    value = score(problem, current)

    local_optima = {}

    def is_local_optimum(state: State) -> bool:
        key = tuple(state.placements[p.id] for p in
problem.packages)
        if key not in local_optima:
            local_optima[key] = not
_has_improving_neighbor(problem, state, objective_name)
        return local_optima[key]

    is_stuck = is_local_optimum(current)
    stuck_count = int(is_stuck)
```

```
    history = [{
        "iteration": 0,
        "objective": value,
        "temperature": None,
        "delta_e": None,
        "exp_delta_over_temperature": None,
        "log_exp_delta_over_temperature": None,
        "exp_overflow": False,
        "acceptance_probability": None,
        "accepted": None,
        "candidate_found": True,
        "stuck": is_stuck,
    }]

    iterations = 0
    stop_reason = ""

    while True:
        t = iterations + 1
        temperature = initial_temperature * max(0.0, 1.0 - (t -
1) / max_iterations)

        if temperature == 0:
            stop_reason = "temperature_zero"
            break

        candidate = random_neighbor(problem, current, rng)
        if candidate is None:
            iterations += 1
            history.append({
                "iteration": iterations,
                "objective": value,
                "temperature": temperature,
                "delta_e": None,
                "exp_delta_over_temperature": None,
                "log_exp_delta_over_temperature": None,
                "exp_overflow": False,
                "acceptance_probability": None,
                "accepted": False,
                "candidate_found": False,
                "stuck": is_stuck,
            })
            continue
```

```
        candidate_value = score(problem, candidate)
        delta_e = candidate_value - value
        iterations += 1

        exp_delta_over_temperature, log_exp, exp_overflow =
_exponential_metrics(delta_e, temperature)
        acceptance_probability = 1.0 if delta_e >= 0 else
exp_delta_over_temperature
        accepted = delta_e >= 0 or rng.random() <
acceptance_probability

        if accepted:
            current = candidate
            value = candidate_value

        was_stuck = is_stuck
        is_stuck = is_local_optimum(current)
        if is_stuck and not was_stuck:
            stuck_count += 1

        history.append({
            "iteration": iterations,
            "objective": value,
            "temperature": temperature,
            "delta_e": delta_e,
            "exp_delta_over_temperature":
exp_delta_over_temperature,
            "log_exp_delta_over_temperature": log_exp,
            "exp_overflow": exp_overflow,
            "acceptance_probability": acceptance_probability,
            "accepted": accepted,
            "candidate_found": True,
            "stuck": is_stuck,
        })

    duration = perf_counter() - started

    return SearchResult(
        algorithm="sa",
        parameters={
            "initial_temperature": initial_temperature,
            "max_iterations": max_iterations,
            "schedule": "linear",
```

```
            "objective": objective_name,
        },
        seed=seed,
        initial_state=initial,
        final_state=current.copy(),
        final_objective=value,
        duration_seconds=duration,
        iterations=iterations,
        history=history,
        stop_reason=stop_reason,
        metadata={
            "stuck_count": stuck_count,
            "stuck_definition": "Entry episodes into states
with no strictly improving legal neighbor, including the
initial state",
        },
    )
```

Fungsi `_has_improving_neighbor()` memeriksa keberadaan neighbor valid yang meningkatkan fungsi objektif terpilih. Untuk dua evaluator yang tersedia, rotasi dan perpindahan paket yang tetap dimuat tidak mengubah himpunan muatan. Oleh karena itu, pemeriksaan peningkatan difokuskan pada pemasukan paket dari luar dan pertukaran paket luar dengan paket dalam yang kontribusinya lebih rendah.

Pemasukan mencoba orientasi paket saat ini pada setiap truk, dengan ketinggian dasar di lantai atau permukaan atas paket non-fragile. Posisi yang memenuhi batas dimensi dan kapasitas kemudian diperiksa menggunakan `is_valid()` serta evaluator. Hasil pemeriksaan optimum lokal disimpan dalam cache berdasarkan seluruh Placement agar state yang sama tidak perlu diperiksa ulang.

Field stuck bernilai True ketika state saat ini tidak memiliki neighbor valid dengan nilai objektif lebih tinggi. Nilai `stuck_count` menghitung episode masuk ke kondisi tersebut, termasuk state awal apabila sudah memenuhi kondisi. Hitungan bertambah pada perubahan status dari False menjadi True; tetap berada pada kondisi stuck tidak menambah hitungan setiap iterasi.

Ukuran ini juga dapat mencakup state optimum global dan tidak menghitung banyaknya state optimum yang berbeda. SA tetap dapat menerima perpindahan setara atau lebih buruk untuk meninggalkan kondisi tersebut. Keluaran `final_state` berupa state terakhir, sehingga nilai akhirnya dapat lebih rendah daripada nilai terbaik yang pernah dilewati.

2.7 Genetic Algorithm

Genetic Algorithm memulai pencarian dari populasi state yang dibangkitkan secara acak. Generasi berikutnya dibentuk melalui seleksi berdasarkan fitness, crossover antara orang tua, dan mutasi [1].

Genetic Algorithm diimplementasikan pada src/algorithms/`genetic_algorithm.py` dengan satu State sebagai individu. Gen setiap paket berupa Placement yang memuat posisi, orientasi, dan `truck_index`. Evaluasi fitness mengikuti pilihan objective melalui `get_objective()`, dengan value sebagai pilihan bawaan.

Populasi awal terdiri atas `population_size` state acak yang masing-masing dibangkitkan menggunakan seed turunan. Parameter `population_size` minimal dua dan `max_iterations` minimal satu, dengan nilai bawaan 20 individu serta 50 generasi. Pada SearchResult, `initial_state` merupakan individu terbaik pada generasi nol, sedangkan seluruh individu generasi nol disimpan pada `initial_population`.

Fungsi `_roulette_select()` memilih parent berdasarkan proporsi fitness. Jika total fitness nol, parent dipilih secara acak dari populasi. Pemilihan dilakukan dua kali sehingga kedua parent dapat berupa individu yang sama.

Fungsi `_crossover()` melakukan crossover satu titik dengan peluang 0,8 berdasarkan urutan paket pada Problem. Jika jumlah paket kurang dari dua atau crossover tidak dilakukan, kedua parent disalin. Anak hasil crossover diperbaiki melalui `_repair_child()`, yang dimulai dari `empty_state()`. Gen paket yang dimuat dicoba menurut urutan z dari bawah ke atas dan hanya dipertahankan jika menghasilkan state valid. Paket yang tidak dapat dipasang tetap berada di luar pada slot parkir kanonik, dengan orientasi warisan.

Fungsi `_mutate()` mencoba mutasi dengan peluang 0,2 per anak melalui `random_neighbor()`. Jika sampling gagal, anak dipertahankan. Setiap generasi menyalin satu individu terbaik sebagai elit, lalu menambahkan anak sampai ukuran populasi terpenuhi. Peluang crossover, peluang mutasi, dan jumlah elit ditetapkan melalui konstanta `CROSSOVER_RATE`, `MUTATION_RATE`, dan ELITISM.

Riwayat mencatat nilai maksimum dan rata-rata fitness mulai dari generasi nol hingga generasi terakhir. Elitisme menjaga nilai maksimum populasi tidak menurun, sedangkan rata-ratanya dapat naik atau turun. Keluaran akhir merupakan individu terbaik yang ditemukan selama pencarian. Cuplikan kode operator dan pencarian sebagai berikut:

```
CROSSOVER_RATE = 0.8
```

```
MUTATION_RATE = 0.2
ELITISM = 1


def _roulette_select(population: list[State], fitnesses:
list[float], total_fitness: float, rng: Random) -> State:
    if total_fitness <= 0:
        return rng.choice(population)
    pick = rng.uniform(0.0, total_fitness)
    current = 0.0
    for individual, fit in zip(population, fitnesses):
        current += fit
        if current >= pick:
            return individual
    return population[-1]


def _repair_child(problem: Problem, raw_placements: dict) ->
State:
    child = empty_state(problem)
    # Ambil gen inside == True, urutkan berdasarkan z menaik
(bawah ke atas)
    inside_candidates = [
        p for p in problem.packages if
raw_placements[p.id].inside
    ]
    inside_candidates.sort(key=lambda p:
raw_placements[p.id].position[2])

    for pkg in inside_candidates:
        candidate = child.copy()
        candidate.placements[pkg.id] = raw_placements[pkg.id]
        if is_valid(problem, candidate):
            child = candidate

    # Gen outside tetap di luar, mempertahankan orientasi yang
diwarisi
    for pkg in problem.packages:
        if not child.placements[pkg.id].inside:
            orientation = raw_placements[pkg.id].orientation
            child.placements[pkg.id] =
replace(child.placements[pkg.id], orientation=orientation)
    return child
```

```
def _crossover(problem: Problem, parent_a: State, parent_b:
State, rng: Random) -> tuple[State, State]:
    packages = problem.packages
    n = len(packages)
    if n < 2 or rng.random() >= CROSSOVER_RATE:
        return parent_a.copy(), parent_b.copy()

    k = rng.randint(1, n - 1)
    raw1 = {}
    raw2 = {}
    for i, p in enumerate(packages):
        if i < k:
            raw1[p.id] = parent_a.placements[p.id]
            raw2[p.id] = parent_b.placements[p.id]
        else:
            raw1[p.id] = parent_b.placements[p.id]
            raw2[p.id] = parent_a.placements[p.id]

    child1 = _repair_child(problem, raw1)
    child2 = _repair_child(problem, raw2)
    return child1, child2


def _mutate(problem: Problem, individual: State, rng: Random)
-> State:
    if rng.random() < MUTATION_RATE:
        mutant = random_neighbor(problem, individual, rng)
        if mutant is not None:
            return mutant
    return individual


def solve(problem: Problem, config: dict | None = None, seed:
int | None = None) -> SearchResult:
    config = dict(config or {})
    unknown = set(config) - {"population_size",
"max_iterations", "objective"}
    if unknown:
        raise ValueError(f"Unknown GA parameters:
{sorted(unknown)}")

    objective_name = config.get("objective", "value")
    objective = get_objective(objective_name)
```

```
    population_size = config.get("population_size", 20)
    max_iterations = config.get("max_iterations", 50)

    integer(population_size, "population_size", 2)
    integer(max_iterations, "max_iterations", 1)

    rng = Random(seed)
    started = perf_counter()

    # Inisialisasi populasi awal (generasi 0)
    population = [random_state(problem, rng.randint(0, 10**9))
for _ in range(population_size)]
    initial_population = [s.copy() for s in population]

    fitnesses = [objective(problem, s) for s in population]
    max_fit = max(fitnesses)
    mean_fit = sum(fitnesses) / len(fitnesses)

    best_individual =
population[fitnesses.index(max_fit)].copy()
    initial_best = best_individual.copy()

    history = [{
        "iteration": 0,
        "objective": max_fit,
        "max_objective": max_fit,
        "mean_objective": mean_fit,
    }]

    for gen in range(1, max_iterations + 1):
        total_fit = sum(fitnesses)
        new_population = [best_individual.copy()]  # Elitisme

        while len(new_population) < population_size:
            p1 = _roulette_select(population, fitnesses,
total_fit, rng)
            p2 = _roulette_select(population, fitnesses,
total_fit, rng)

            c1, c2 = _crossover(problem, p1, p2, rng)
            c1 = _mutate(problem, c1, rng)
            c2 = _mutate(problem, c2, rng)

            new_population.append(c1)
```

```
            if len(new_population) < population_size:
                new_population.append(c2)

        population = new_population
        fitnesses = [objective(problem, s) for s in population]
        gen_max = max(fitnesses)
        gen_mean = sum(fitnesses) / len(fitnesses)

        if gen_max > objective(problem, best_individual):
            best_individual =
population[fitnesses.index(gen_max)].copy()

        history.append({
            "iteration": gen,
            "objective": max(gen_max, objective(problem,
best_individual)),
            "max_objective": max(gen_max, objective(problem,
best_individual)),
            "mean_objective": gen_mean,
        })

    duration = perf_counter() - started
    final_obj = objective(problem, best_individual)

    return SearchResult(
        algorithm="ga",
        parameters={
            "population_size": population_size,
            "max_iterations": max_iterations,
            "objective": objective_name,
        },
        seed=seed,
        initial_state=initial_best,
        final_state=best_individual.copy(),
        final_objective=final_obj,
        duration_seconds=duration,
        iterations=max_iterations,
        history=history,
        initial_population=initial_population,
        stop_reason="max_iterations",
```

```
        metadata={
            "elitism": ELITISM,
            "crossover_rate": CROSSOVER_RATE,
            "mutation_rate": MUTATION_RATE,
        },
    )
```

2.8 Pengembangan Multi-Truk

Bonus multi-truk memperluas persoalan agar program dapat menerima lebih dari satu truk dan memaksimalkan total nilai paket yang diangkut seluruh armada [2]. Pengembangan multi-truk menggunakan Problem.trucks untuk menyimpan satu atau beberapa truk. Input JSON dapat memuat daftar trucks dengan dimensi dan kapasitas masing-masing. Format lama berupa truck tetap didukung, dengan `num_trucks` bernilai bawaan satu; jika jumlahnya lebih dari satu, spesifikasi truk yang sama digunakan untuk seluruh truk tersebut.

Setiap paket mempunyai satu Placement sehingga paket hanya dapat dimuat pada satu truk. Nilai `truck_index` menunjukkan indeks truk, sedangkan None menyatakan paket berada di luar. Posisi paket menggunakan koordinat lokal truk tujuan. Jumlah truk ditentukan melalui input dan tetap selama pencarian.

Validasi batas dinding, kapasitas berat, overlap, support, dan fragile dilakukan untuk tiap truk. Dua paket pada truk berbeda dapat mempunyai koordinat lokal yang sama dan tidak dianggap bertabrakan. Paket hanya dapat ditopang oleh paket pada truk yang sama. Paket di luar harus berada sepenuhnya di luar batas setiap truk.

Operator `move_package()` dapat mengganti posisi sekaligus indeks truk tujuan. Operator `swap_packages()` menukar posisi dan indeks truk dua paket, dengan mempertahankan orientasi masing-masing. Pembangkitan state awal, enumerasi neighbor, dan sampling kandidat menggunakan daftar truk yang sama. Perpindahan antartruk dapat menyediakan ruang bagi paket lain, meskipun perpindahan itu sendiri belum meningkatkan nilai objektif.

Fungsi objektif menjumlahkan kontribusi paket yang dimuat di seluruh truk, sesuai evaluator value atau `eta_priority`. Truk kosong tidak dikenai penalti, sehingga program tidak meminimalkan jumlah truk yang digunakan dan tidak menghitung rute perjalanan. Algoritma pencarian menggunakan model, validator, dan pembangkit kandidat bersama untuk menangani satu maupun beberapa truk.

Fungsi `coordinate_map()` menghasilkan peta terpisah untuk setiap truk dengan urutan indeks [truck][z][y][x]. Peta ini digunakan pada keluaran terminal dan disimpan bersama hasil pencarian.

2.9 Menu, penyimpanan, dan visualisasi

Program dijalankan melalui src/main.py. Setelah membaca file input JSON, pengguna memilih fungsi objektif value atau `eta_priority`, kemudian memilih algoritma. Menu 1, 2, dan 3 menjalankan satu pencarian HC, SA, dan GA; menu 5, 6, dan 7 menjalankan Sideways, Random Restart, dan Stochastic Hill-Climbing. Fungsi `run_search()` menghubungkan pilihan dengan solver dan memeriksa kesesuaian `final_objective` terhadap state akhir.

Menu 4 menjalankan tiga pencarian HC dengan tiga seed berbeda. Jenis HC dan parameternya dapat dipilih untuk setiap pencarian. Menu 8 menjalankan tiga pencarian SA, sedangkan menu 9 menjalankan 18 pencarian GA: sembilan untuk variasi generasi dengan populasi tetap dan sembilan untuk variasi populasi dengan generasi tetap. Parameter eksperimen dimasukkan melalui terminal.

Terminal menampilkan state awal dan akhir, meliputi ID paket, status muatan, nomor truk, posisi, orientasi, ukuran setelah rotasi, value, berat, fragile, serta ETA. Peta koordinat ditampilkan per truk dan per lapisan z. Ringkasan hasil memuat nilai objektif terpilih, iterasi, durasi, alasan berhenti, dan metadata khusus algoritma.

Fungsi `save_result()` pada src/core/io.py menyimpan JSON berisi data problem, parameter, seed, state awal dan akhir, riwayat, metadata, populasi awal GA, serta peta koordinat. Data problem disimpan dalam format daftar trucks dan penempatan menyimpan `truck_index`. JSON ini dapat digunakan kembali untuk visualisasi tanpa menjalankan pencarian.

Visualisasi pada src/visualization.py menggunakan `box_faces()` dan Poly3DCollection untuk menggambar paket sebagai balok. Fungsi `build_result_figure()` menyusun dua kolom untuk state awal dan akhir, dengan satu baris untuk setiap truk serta satu baris tambahan bagi paket di luar. Untuk K truk, jumlah panel adalah $2 \times (K + 1)$. Setiap panel truk menampilkan dimensi, jumlah paket, dan berat terhadap kapasitasnya.

Paket di luar ditata dalam grid tampilan agar mudah dilihat. Penataan ini tidak mengubah koordinat pada state; posisi aslinya ditampilkan pada detail paket. Warna setiap paket dipertahankan antara state awal dan akhir, meskipun warna dapat berulang setelah 20 paket. Paket fragile ditandai garis putus-putus serta label [F] ketika dipilih.

Jendela Tk menyediakan rotasi dengan seret, zoom menggunakan scroll atau tombol +/−, tombol R untuk mengembalikan tampilan, toolbar, dan scrollbar untuk melihat panel lain. Klik pada balok menampilkan ID, posisi asli, ukuran, orientasi, value, berat, dan ETA. Perintah menjalankan program dan membuka hasil sebagai berikut:

```
python src/main.py
python -m src.visualization hasil.json
python -m src.visualization hasil.json --save state_3d.png
--no-show
```

Opsi --save menyimpan gambar PNG, sedangkan --no-show mengekspor gambar tanpa membuka jendela dan harus disertai --save. Pada satu pencarian melalui menu, pengguna menyimpan JSON terlebih dahulu, kemudian dapat memilih pembuatan grafik serta gambar 3D dan pembukaan jendela visualisasi. Jika jendela tidak dapat dibuka, JSON yang sudah tersimpan tetap tersedia.

Grafik HC menampilkan nilai objektif terhadap iterasi; untuk satu pencarian Random Restart, grafik juga memuat `best_objective` dan penanda restart. Grafik SA menampilkan nilai objektif serta exp(ΔE/T) pada skala logaritmik, menggunakan ΔE/T agar eksponensial yang melampaui jangkauan float tetap dapat ditampilkan. Kandidat yang tidak ditemukan tidak menghasilkan titik eksponensial. Grafik GA menampilkan nilai maksimum dan rata-rata fitness populasi.

Batch eksperimen menyimpan JSON setiap pencarian, grafik metrik, dan rekapitulasi summary.csv. Gambar serta jendela 3D dapat dibuat kembali dari JSON hasil melalui perintah visualisasi.

BAB 3
Eksperimen dan Analisis

3.1 Konfigurasi Eksperimen

Spesifikasi tugas mengatur tiga kali pengulangan untuk algoritma selain Genetic Algorithm. Pengujian Genetic Algorithm dilakukan melalui tiga variasi jumlah generasi dengan populasi tetap dan tiga variasi jumlah populasi dengan generasi tetap, masing-masing diulang tiga kali [2].

Eksperimen dilakukan untuk mengevaluasi kualitas solusi, durasi pencarian, dan konsistensi hasil algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Pengujian juga mencakup variasi Hill-Climbing, penggunaan beberapa truk, serta fungsi objektif alternatif berbasis ETA.

Eksperimen dibagi menjadi tiga skenario, yaitu `single_value`, `multi_value`, dan `single_eta`. Setiap konfigurasi dijalankan menggunakan tiga seed berbeda, yaitu 11, 22, dan 33. Secara keseluruhan, terdapat 54 run yang tersimpan dalam bentuk JSON, grafik, dan rekapitulasi CSV.

3.1.1 Dataset dan Skenario Pengujian

Skenario `single_value` menggunakan satu truk dan fungsi objektif berupa total nilai paket yang diangkut. Skenario ini digunakan untuk membandingkan algoritma utama dan variasi Hill-Climbing.

Skenario `multi_value` menggunakan tiga truk dengan dimensi dan kapasitas berbeda. Fungsi objektifnya memaksimalkan total nilai paket yang diangkut seluruh armada. Skenario `single_eta` menggunakan persoalan satu truk yang sama dengan `single_value`, tetapi fungsi objektif diubah menjadi jumlah nilai paket yang dibobot berdasarkan ETA.

Tabel 3.1 Skenario eksperimen

Skenario	Jumlah truk	Jumlah paket	Fungsi objektif	Algoritma yang diuji	Jumlah run
single_value	1	12	Total value	Empat varian HC, SA, dan GA	33
multi_value	3	15	Total value armada	Empat varian HC, SA, dan GA	18

single_eta	1	12	Total value berbobot ETA	Steepest Ascent HC	3
Total					54

Dimensi dan kapasitas truk pada masing-masing skenario ditunjukkan pada Tabel 3.2. Dimensi dinyatakan dalam urutan lebar, panjang, dan tinggi, sedangkan kapasitas merupakan batas berat muatan.

Tabel 3.2 Konfigurasi truk

Skenario	**Truk**	**Dimensi $w \times l \times h$**	**Kapasitas berat**
single_value dan single_eta	1	10 × 20 × 5	500
multi_value	1	5 × 6 × 4	150
multi_value	2	6 × 8 × 4	250
multi_value	3	8 × 10 × 5	400

Dataset satu truk dan multi-truk memiliki jumlah paket serta konfigurasi armada yang berbeda. Oleh karena itu, perbedaan nilai akhir kedua skenario tidak dapat diartikan sebagai pengaruh penambahan jumlah truk saja. Perbandingan antaralgoritma dilakukan dalam skenario yang sama.

3.1.2 Konfigurasi Hill-Climbing

Empat varian Hill-Climbing diuji pada skenario single_value dan multi_value. Setiap varian dijalankan tiga kali menggunakan seed 11, 22, dan 33. Steepest Ascent juga dijalankan tiga kali pada skenario single_eta.

Tabel 3.3 Parameter eksperimen Hill-Climbing

Varian	**Batas iterasi**	**Parameter tambahan**
Steepest Ascent	Tanpa batas eksplisit	-
Sideways Move	Tanpa batas eksplisit	max_sideways_moves = 20
Random Restart	Tanpa batas eksplisit per run internal	max_restarts = 3

Stochastic	1000	-

Tidak adanya batas iterasi eksplisit tidak berarti pencarian berjalan tanpa kondisi penghentian. Penghentian mengikuti mekanisme masing-masing varian, seperti tidak ditemukannya peningkatan atau tercapainya batas sideways.

Pada Random Restart, tiga restart berarti satu run awal dan tiga run tambahan dalam setiap eksekusi eksperimen. Jumlah restart tersebut dibedakan dari tiga pengulangan eksperimen berdasarkan seed.

3.1.3 Konfigurasi Simulated Annealing

Simulated Annealing diuji pada skenario single_value dan multi_value. Setiap skenario dijalankan tiga kali dengan parameter yang sama dan seed berbeda.

Tabel 3.6 Parameter eksperimen Simulated Annealing

Parameter	Nilai
Temperatur awal	100
Maksimum iterasi	1000
Jadwal penurunan temperatur	Linear
Seed	11, 22, 33
Jumlah run per skenario	3

Parameter temperatur dan iterasi dipertahankan sama pada kedua skenario. Data yang dicatat mencakup nilai objektif awal dan akhir, nilai terbaik dalam riwayat, durasi, perkembangan objective, informasi penerimaan kandidat, dan jumlah episode stagnasi sesuai definisi implementasi.

3.1.4 Konfigurasi Genetic Algorithm

Eksperimen variasi parameter Genetic Algorithm dilakukan pada skenario `single_value` dalam dua kelompok. Kelompok A menggunakan jumlah populasi tetap dan memvariasikan jumlah generasi. Kelompok B menggunakan jumlah generasi tetap dan memvariasikan jumlah populasi.

Tabel 3.7 Konfigurasi Genetic Algorithm pada skenario satu truk

Kelompok	Parameter kontrol	Variasi parameter	Seed tiap konfigurasi	Jumlah run
A	Populasi 20	Generasi 25, 50, 100	11, 22, 33	9
B	Generasi 50	Populasi 10, 20, 40	11, 22, 33	9

Terdapat 18 run GA pada skenario satu truk. Kombinasi populasi 20 dan generasi 50 muncul pada kedua kelompok. Pengulangan kombinasi dengan seed yang sama tersebut tidak diperlakukan sebagai konfigurasi baru atau tambahan seed independen dalam analisis.

Pada skenario `multi_value`, GA dijalankan tiga kali menggunakan populasi 20, jumlah generasi 50, serta seed 11, 22, dan 33. Pengujian ini digunakan untuk mengevaluasi penerapan GA pada multi-truk. Analisis pengaruh variasi populasi dan generasi menggunakan hasil eksperimen satu truk.

3.1.5 Pencatatan dan Penyimpanan Hasil

Seluruh hasil eksperimen disimpan pada direktori output/experiments/. Setiap JSON hasil memuat konfigurasi persoalan, parameter algoritma, seed, state awal dan akhir, nilai objektif, durasi, jumlah iterasi, riwayat pencarian, dan metadata algoritma.

Rekapitulasi seluruh run tersedia dalam summary.csv. Data yang digunakan dalam analisis meliputi:

1. Nilai objektif awal, akhir, dan terbaik yang tercatat selama pencarian.
2. Durasi pencarian dalam detik.
3. Jumlah iterasi dan alasan penghentian.
4. Jumlah paket serta total berat yang diangkut.
5. Jumlah sideways, restart, iterasi per restart, atau stagnasi sesuai algoritma.
6. Jumlah populasi dan generasi pada GA.

3.2 Hasil pencarian dan kedekatan terhadap optimum

Tabel 3.2 Nilai objektif akhir pada kasus satu truk. GA menggunakan konfigurasi A50. Rerata dan simpangan baku populasi (SD) dihitung dari tiga seed.

Tabel 3.2 Hasil nilai objektif dari run setiap algoritma

Algoritma	Seed 11	Seed 22	Seed 33	Rerata	SD
Steepest Ascent	1095	1245	1245	1195,00	70,71
Sideways	1095	1245	1245	1195,00	70,71
Random Restart	1245	1245	1245	1245,00	0,00
Stochastic	1015	1245	1245	1168,33	108,42
SA	890	1150	1185	1075,00	131,59
GA A50	1125	1185	1175	1161,67	26,25

Steepest Ascent menghasilkan nilai 1095, 1245, dan 1245, dengan tiga pemindaian neighborhood pada setiap seed. Semua run berhenti karena `no_improving_neighbor`. Sideways menghasilkan nilai akhir yang sama, tetapi melakukan 22 iterasi dan menerima total 20 perpindahan setara pada setiap seed. Pada percobaan ini, penelusuran plateau tidak memperbaiki hasil Steepest Ascent.

Random Restart menghasilkan 1245 pada seluruh seed. Jumlah iterasinya adalah 14, 17, dan 20, yang merupakan penjumlahan empat pencarian. Pada seed 11, tambahan state awal membantu mencapai nilai 1245 setelah pencarian pertama berhenti pada 1095. Stochastic menghasilkan 1015, 1245, dan 1245 setelah 1000 iterasi, sehingga hasilnya lebih bervariasi meskipun biaya sampling per iterasi lebih kecil.

SA menghasilkan nilai akhir 890, 1150, dan 1185. Nilai terbaik yang dilewati pada ketiga riwayat adalah 995, 1150, dan 1185. Pada seed 11, hasil akhir 890 lebih rendah daripada nilai awal 900 maupun rekor 995. Solver mengembalikan state terakhir setelah pendinginan, sehingga penerimaan penurunan dapat memengaruhi nilai akhir. Seluruh run SA berhenti dengan `temperature_zero`.

Batas atas nilai untuk `single_value` adalah 1245, yaitu jumlah nilai seluruh 12 paket. Solusi valid yang memuat seluruh paket mencapai batas tersebut, sehingga optimum global 1245 terbukti untuk input ini. Random Restart mencapainya pada tiga dari tiga seed; Steepest Ascent,

Sideways, dan Stochastic pada dua dari tiga seed. SA serta GA A50 belum mencapai optimum pada ketiga seednya.

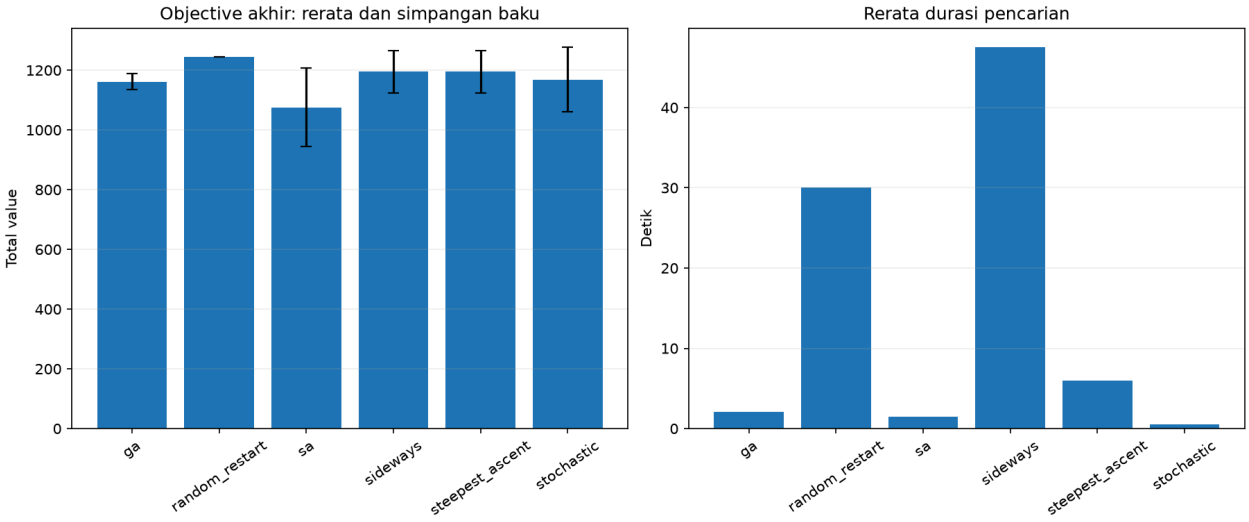
Selisih rerata akhir terhadap optimum adalah 50 untuk Steepest Ascent dan Sideways, 76,67 untuk Stochastic, 170 untuk SA, serta 83,33 untuk GA A50. Selisih ini berlaku untuk dataset satu truk tersebut dan tidak menyatakan peringkat umum algoritma.

Tabel 3.3 Nilai objektif akhir pada kasus multi-truk. GA memakai populasi 20 dan 50 generasi.

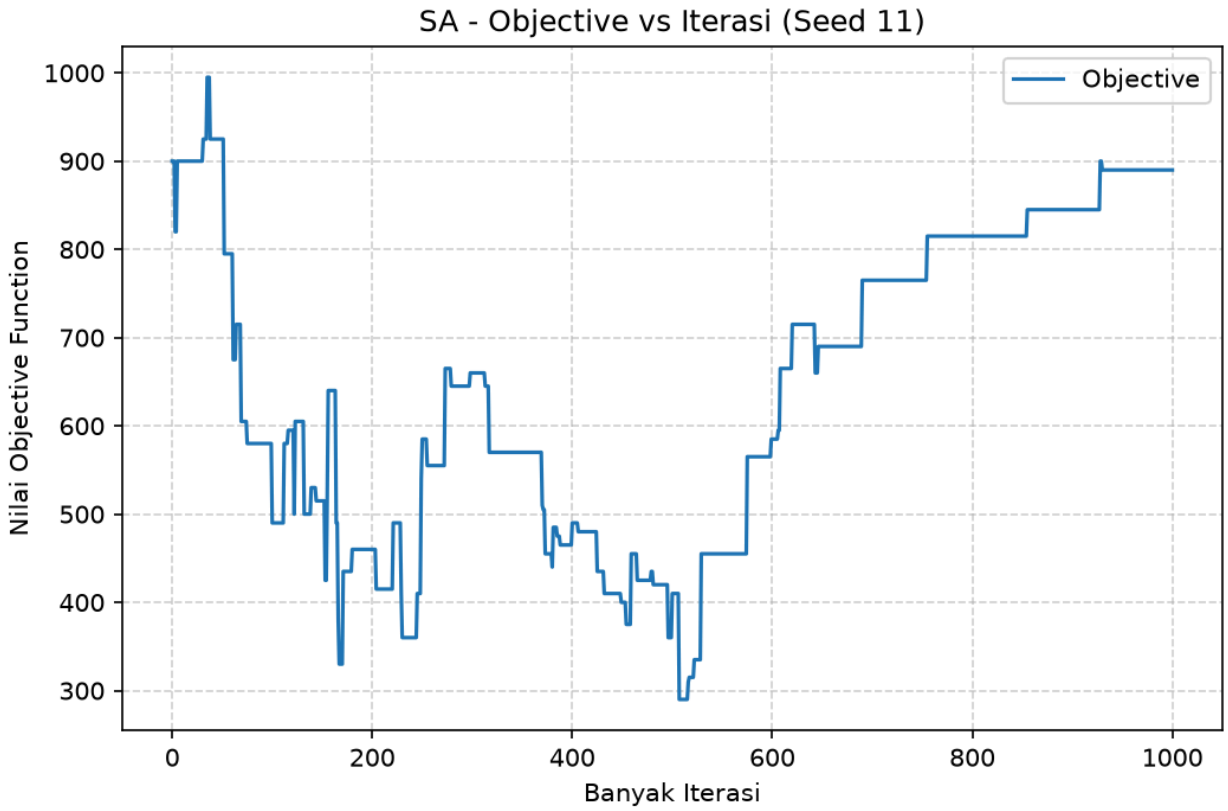
Algoritma	Seed 11	Seed 22	Seed 33	Rerata	SD
Steepest Ascent	1285	1285	1285	1285,00	0,00
Sideways	1285	1285	1285	1285,00	0,00
Random Restart	1285	1285	1285	1285,00	0,00
Stochastic	1285	1165	1245	1231,67	49,89
SA	1130	1100	1110	1113,33	12,47
GA (20, 50)	1175	1220	1225	1206,67	22,48

Steepest Ascent, Sideways, dan Random Restart memuat seluruh 15 paket pada ketiga seed. Nilai 1285 sama dengan jumlah nilai seluruh paket, sehingga optimum global juga terbukti untuk kasus multi-truk ini. Stochastic mencapai 1285 pada seed 11, sedangkan hasil seed 22 dan 33 adalah 1165 dan 1245. GA menghasilkan 1175, 1220, dan 1225; SA menghasilkan 1130, 1100, dan 1110.

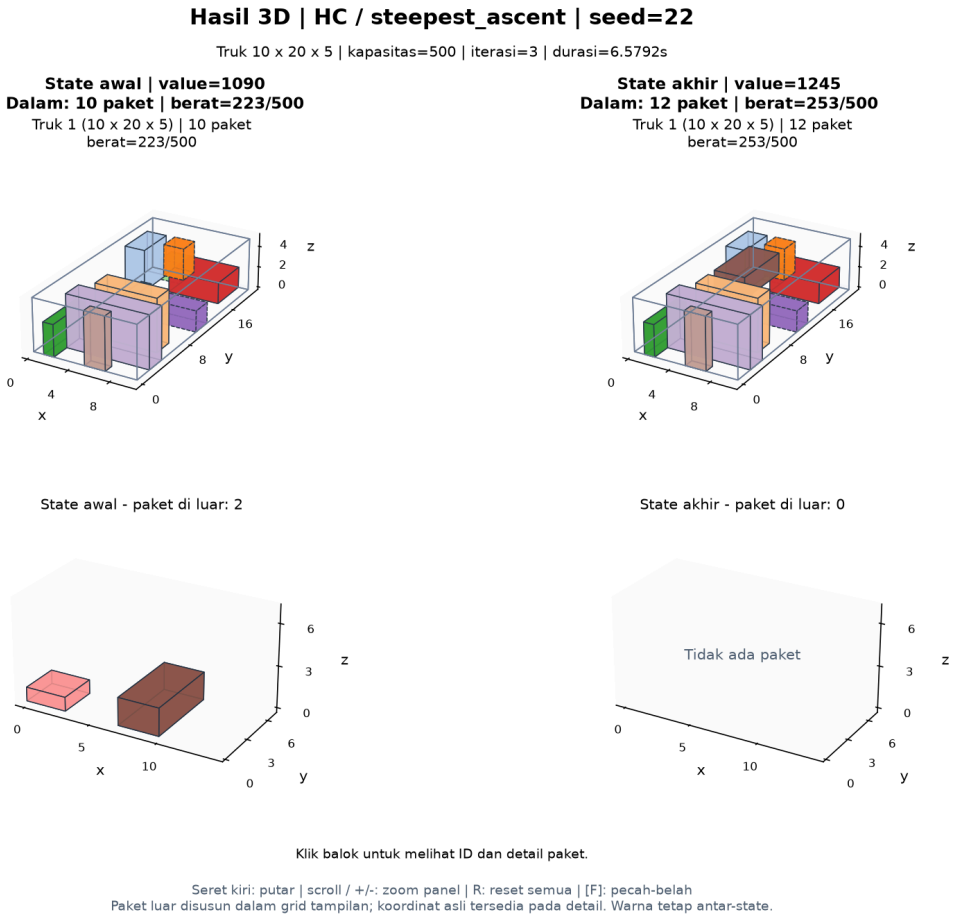
Pada fungsi objektif `eta_priority`, Steepest Ascent menghasilkan 332,666667, 362,666667, dan 362,666667. Nilai value tanpa pembobotan pada solusi tersebut adalah 1095, 1245, dan 1245. Jumlah kontribusi seluruh paket sebesar 362,666667 tercapai pada seed 22 dan 33. Nilai ETA berbobot dan value mempunyai skala berbeda, sehingga keduanya tidak dibandingkan sebagai satu ukuran kualitas.



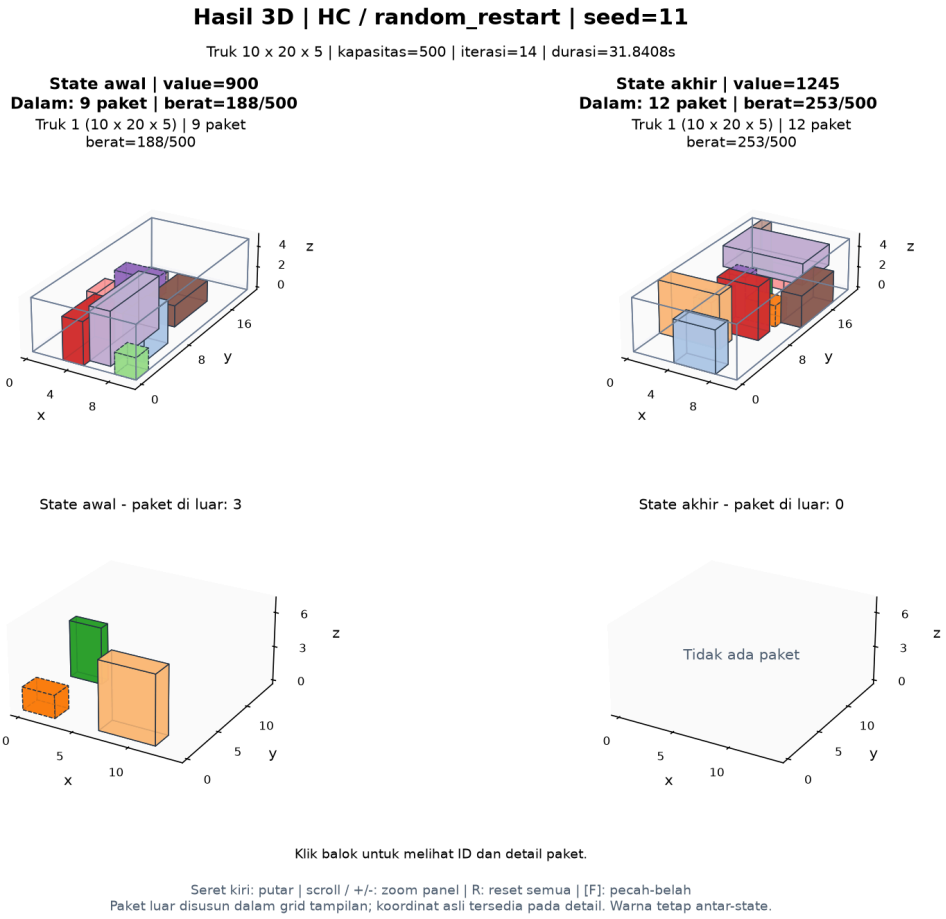
Gambar 3.1 Perbandingan rerata nilai akhir, simpangan baku populasi, dan durasi



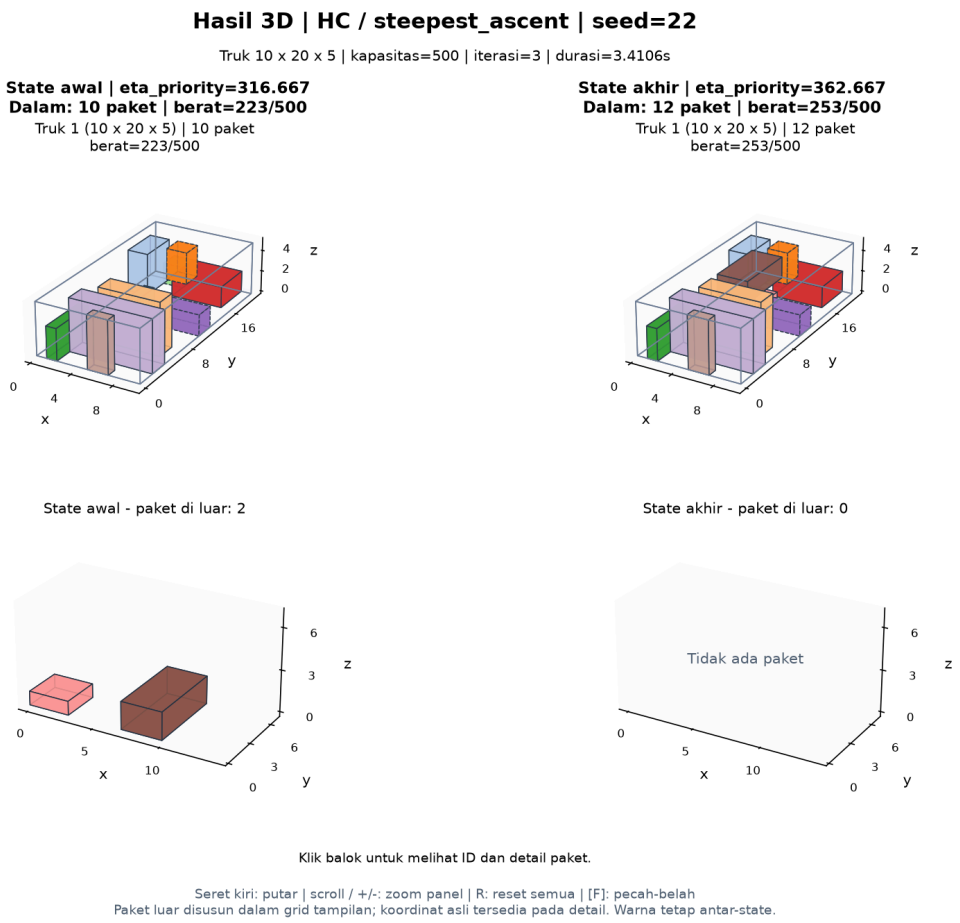
Gambar 3.2 Riwayat SA seed 11 pada single_value



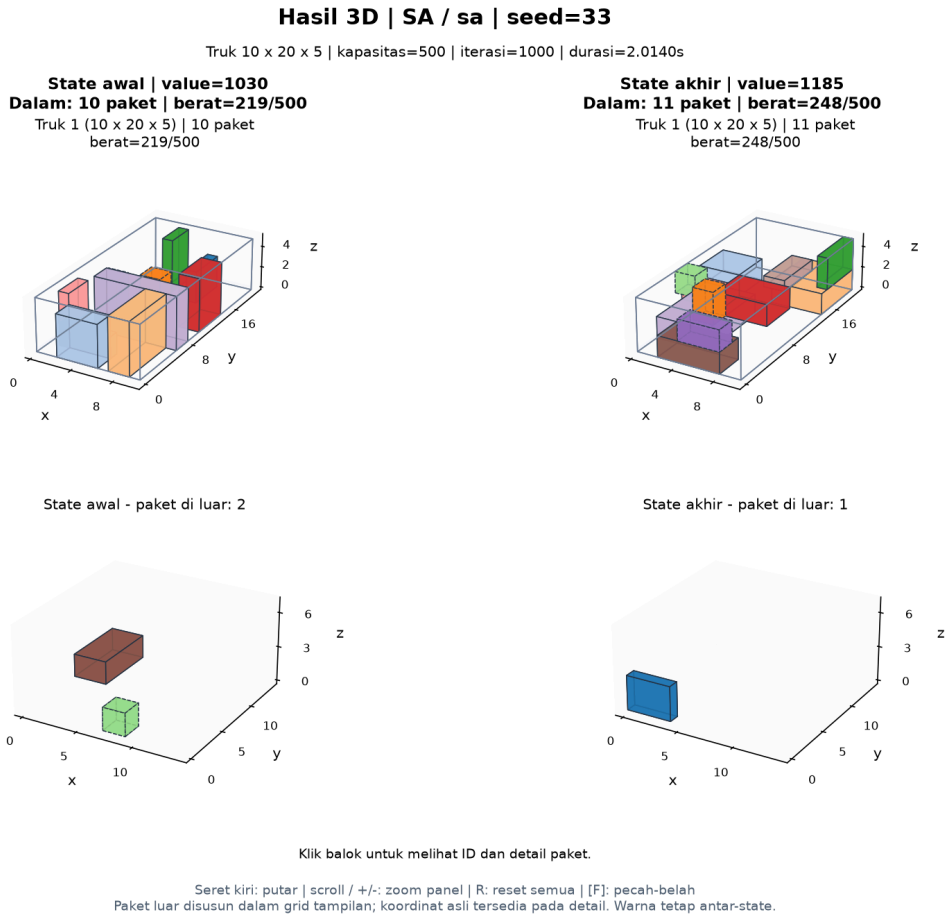
Gambar 3.3 State awal dan akhir Steepest Ascent seed 22 pada single_value



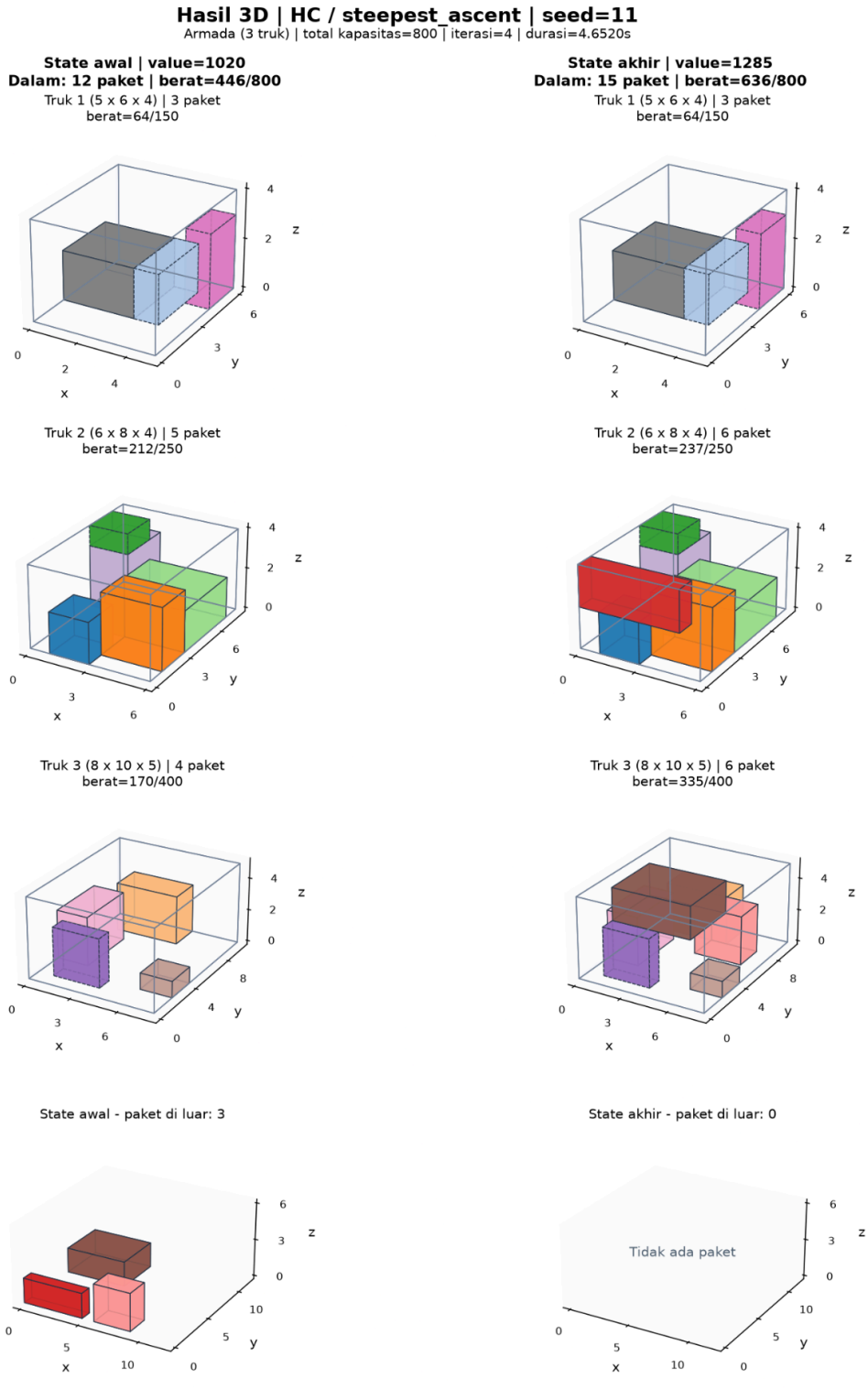
Gambar 3.4 State awal dan hasil terbaik Random Restart seed 11 pada single_value



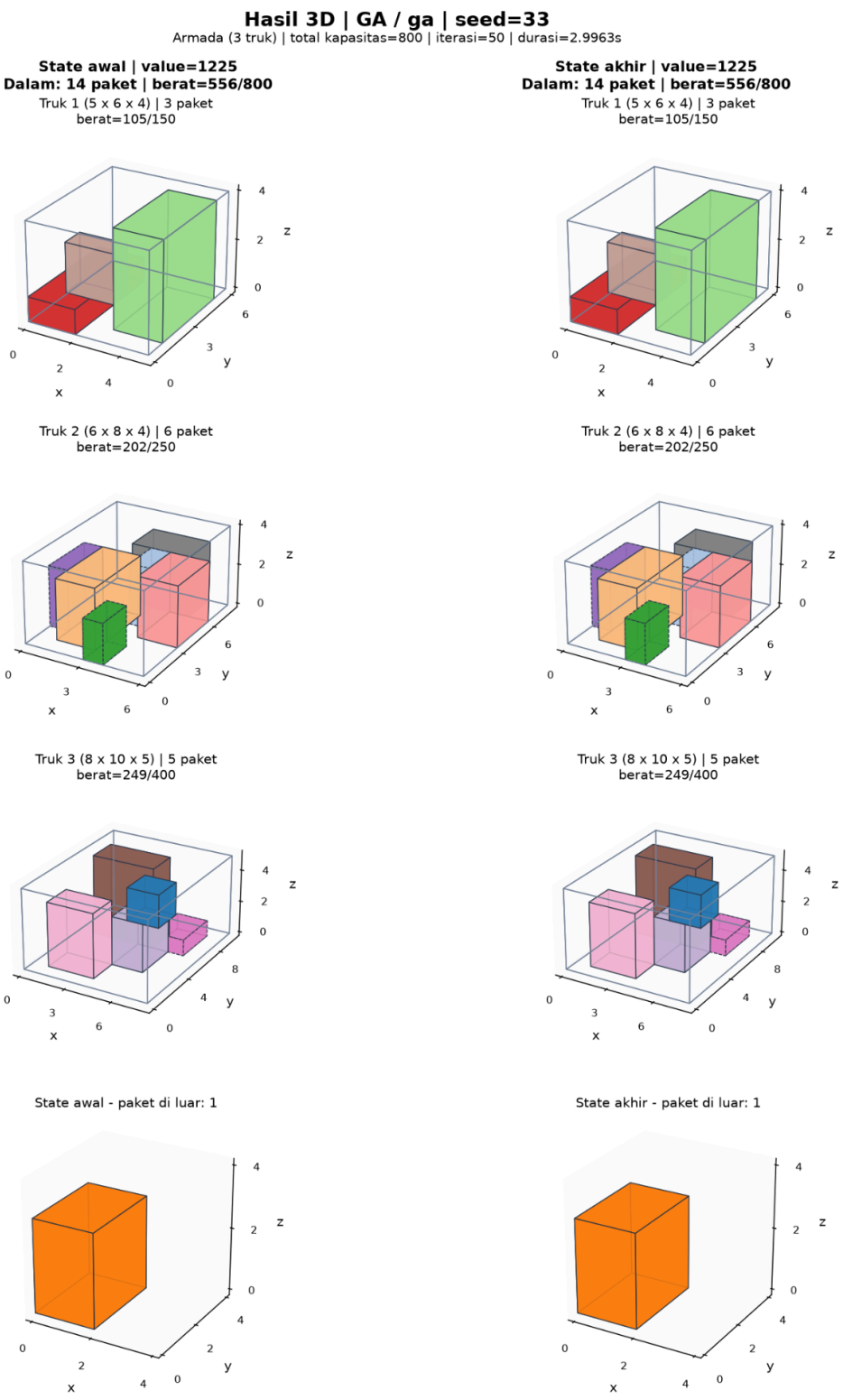
Gambar 3.5 Steepest Ascent seed 22 dengan eta_priority



Gambar 3.6 State awal dan akhir SA seed 33 pada single_value


Hasil 3D | HC / steepest_ascent | seed=11
Armada (3 truk) | total kapasitas=800 | iterasi=4 | durasi=4.6520s
State awal | value=1020
Dalam: 12 paket | berat=446/800
Truk 1 (5 x 6 x 4) | 3 paket
berat=64/150
Truk 2 (6 x 8 x 4) | 5 paket
berat=212/250
Truk 3 (8 x 10 x 5) | 4 paket
berat=170/400
State awal - paket di luar: 3
State akhir | value=1285
Dalam: 15 paket | berat=636/800
Truk 1 (5 x 6 x 4) | 3 paket
berat=64/150
Truk 2 (6 x 8 x 4) | 6 paket
berat=237/250
Truk 3 (8 x 10 x 5) | 6 paket
berat=335/400
State akhir - paket di luar: 0
Tidak ada paket
Klik balok untuk melihat ID dan detail paket.
Seret kiri: putar | scroll / +/-: zoom panel | R: reset semua | [F]: pecah-belah
Paket luar disusun dalam grid tampilan; koordinat asli tersedia pada detail. Warna tetap antar-state.

Gambar 3.7 Steepest Ascent seed 11 pada multi_value


Hasil 3D | GA / ga | seed=33
Armada (3 truk) | total kapasitas=800 | iterasi=50 | durasi=2.9963s
State awal | value=1225
Dalam: 14 paket | berat=556/800
State akhir | value=1225
Dalam: 14 paket | berat=556/800
Truk 1 (5 x 6 x 4) | 3 paket
berat=105/150
Truk 2 (6 x 8 x 4) | 6 paket
berat=202/250
Truk 3 (8 x 10 x 5) | 5 paket
berat=249/400
State awal - paket di luar: 1
State akhir - paket di luar: 1
Klik balok untuk melihat ID dan detail paket.
Seret kiri: putar | scroll / +/-: zoom panel | R: reset semua | [F]: pecah-belah
Paket luar disusun dalam grid tampilan; koordinat asli tersedia pada detail. Warna tetap antar-state.

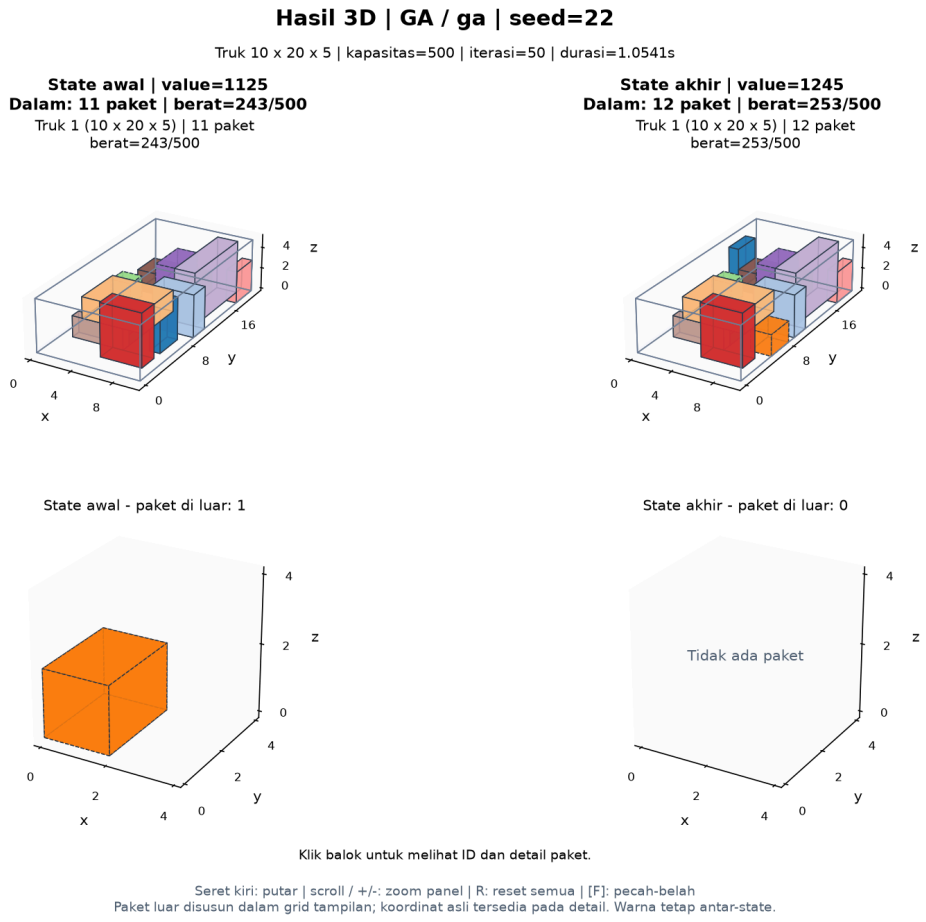
Gambar 3.8 GA seed 33 pada multi_value

Tabel 3.4 Seluruh 18 run GA pada `single_value`. Awal adalah individu terbaik generasi nol, akhir adalah individu terbaik yang dikembalikan, dan rerata akhir adalah rata-rata fitness populasi terakhir.

Konfigurasi	Seed	Awal	Akhir	Rerata akhir	Durasi (s)
A25	11	1095	1125	1039,00	3,2968
A25	22	1125	1135	1015,75	1,6914
A25	33	1090	1150	1036,75	2,9637
A50	11	1095	1125	891,00	1,7779
A50	22	1125	1185	1083,25	2,0351
A50	33	1090	1175	1088,25	2,3730
A100	11	1095	1185	1050,50	2,5388
A100	22	1125	1195	1033,25	2,7636
A100	33	1090	1195	1165,25	3,2502
B10	11	1095	1165	1134,00	0,5544
B10	22	1125	1245	1206,00	1,0541
B10	33	1090	1125	1042,00	1,1972
B20	11	1095	1125	891,00	2,0941
B20	22	1125	1185	1083,25	2,4301
B20	33	1090	1175	1088,25	2,7791
B40	11	1170	1170	1068,25	4,4303
B40	22	1150	1150	947,75	4,4034

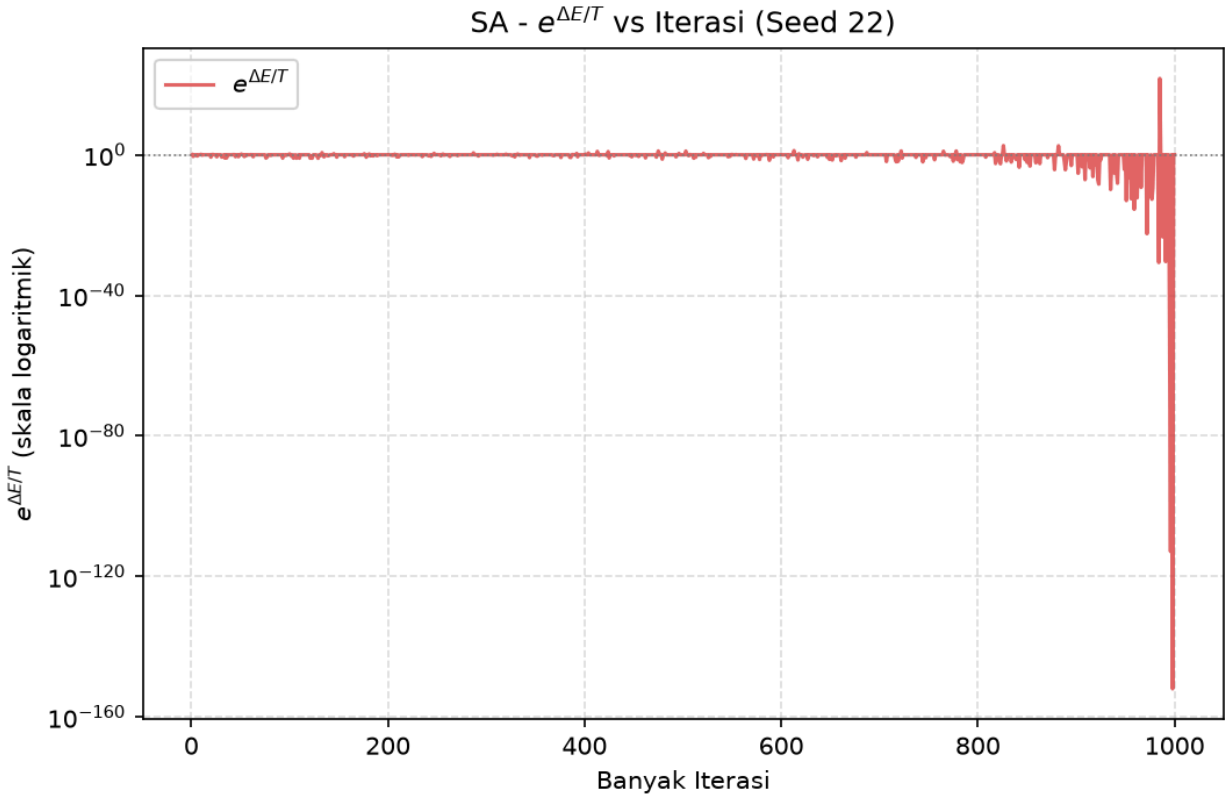
B40	33	1090	1115	1016,38	4,1389

GA mencapai nilai optimum 1245 pada satu dari 18 run yang tersimpan, yaitu B10 seed 22. Konfigurasi lainnya menghasilkan nilai akhir antara 1115 dan 1195. Pengulangan konfigurasi A50 dan B20 memberikan hasil kualitas yang sama pada setiap seed.



Gambar 3.9 State awal terbaik dan hasil GA B10 seed 22 pada single_value

3.3 Probabilitas penerimaan dan optimum lokal SA



Gambar 3.10 Eksponensial mentah exp(ΔE/T) pada SA seed 22 untuk single_value

Grafik `plot_sa_probability()` menggunakan eksponen yang tersimpan pada `log_exp_delta_over_temperature` agar exp(ΔE/T) dapat ditampilkan pada skala logaritmik. Nilai tersebut berbeda dari `acceptance_probability`, yang dibatasi pada satu. Pada `single_value` seed 22, eksponensial mentah maksimum sekitar 5,18 × 10^21, sedangkan probabilitas penerimaan maksimumnya tetap satu.

Suhu pada percobaan pertama adalah 100 dan pada percobaan ke-1000 sekitar 0,1. Untuk contoh penurunan ΔE = −10, peluang penerimaan turun dari sekitar 0,9048 pada T = 100 menjadi 3,72 × 10^-44 pada T = 0,1. Kandidat bernilai sama atau lebih tinggi selalu diterima.

Nilai `stuck_count` adalah nol pada seluruh run SA, baik `single_value` maupun `multi_value`. Field ini menghitung episode memasuki state yang tidak mempunyai neighbor valid dengan nilai objektif lebih tinggi, termasuk state awal apabila memenuhi kondisi. Pada riwayat yang tersimpan, pemeriksaan masih menemukan kemungkinan peningkatan pada setiap state yang dikunjungi. Hasil akhir yang belum optimal tersebut tidak dapat disebut optimum lokal berdasarkan pemeriksaan ini.

Kondisi nilai objektif yang lama tidak meningkat tetap dapat terlihat pada grafik, tetapi tidak otomatis menaikkan `stuck_count`. Pada enam run ini tidak terdapat kegagalan sampling maupun overflow eksponensial. Nilai eksponensial yang sangat kecil dapat dibulatkan menjadi nol oleh representasi float.

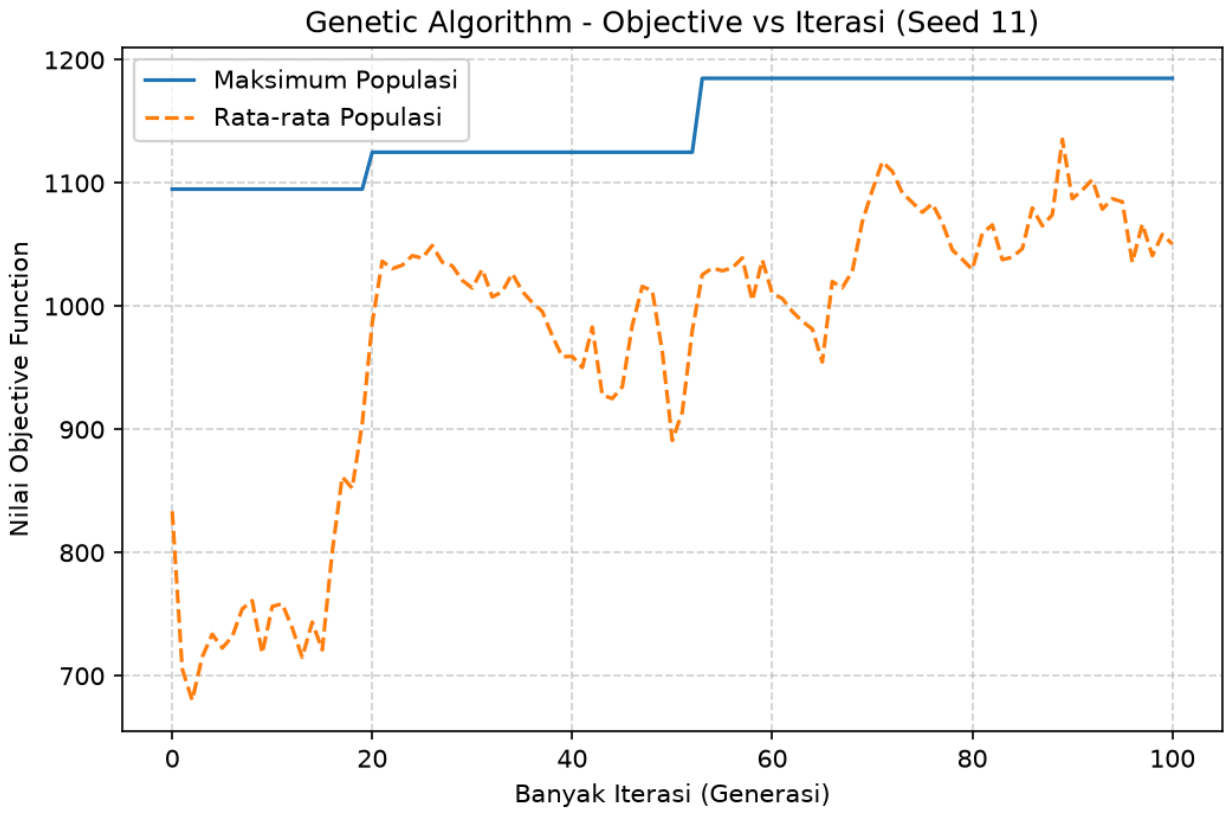
Pada `single_value`, SA menerima 874, 856, dan 844 kandidat untuk seed 11, 22, dan 33. Di antaranya terdapat 38, 46, dan 48 perpindahan yang menurunkan nilai objektif. Jumlah kandidat setara yang diterima adalah 799, 765, dan 736, sehingga sebagian besar langkah mempertahankan nilai objektif.

Pada `multi_value`, jumlah kandidat yang diterima adalah 859, 854, dan 845, dengan 48, 57, dan 51 penerimaan penurunan. Penerimaan langkah setara dan penurunan memungkinkan perubahan susunan, tetapi pada parameter yang diuji belum membawa SA mencapai batas atas kedua dataset.

3.4 Pengaruh parameter Genetic Algorithm

Tabel 3.5 Ringkasan konfigurasi GA pada `single_value`, masing-masing tiga seed. Rerata awal memakai individu terbaik generasi nol; SD memakai tiga nilai akhir.

Konfigurasi	Rerata awal	Rerata akhir	SD	Durasi rerata (s)
A25	1103,33	1136,67	10,27	2,6506
A50	1103,33	1161,67	26,25	2,0620
A100	1103,33	1191,67	4,71	2,8509
B10	1103,33	1178,33	49,89	0,9352
B20	1103,33	1161,67	26,25	2,4344
B40	1136,67	1145,00	22,73	4,3242



Gambar 3.11 Nilai maksimum dan rata-rata fitness GA A100 seed 11

Pada populasi tetap 20, penambahan generasi dari 25 ke 50 dan 100 meningkatkan rerata nilai akhir dari 1136,67 menjadi 1161,67 dan 1191,67. Setiap seed mempertahankan atau meningkatkan hasil terbaiknya ketika jumlah generasi diperpanjang. Nilai akhir A100 adalah 1185, 1195, dan 1195, dengan simpangan baku 4,71, meskipun belum mencapai optimum 1245.

Durasi rerata A25, A50, dan A100 adalah 2,6506, 2,0620, dan 2,8509 detik. A25 tercatat lebih lama daripada A50, sehingga urutan waktu pada pelaksanaan ini tidak mengikuti jumlah generasi secara monoton. Satu pengukuran untuk setiap kombinasi belum memisahkan biaya algoritma dari variasi kondisi eksekusi.

Pada 50 generasi tetap, rerata nilai akhir untuk populasi 10, 20, dan 40 adalah 1178,33, 1161,67, dan 1145,00. Populasi 10 memberikan rerata tertinggi pada kelompok B dan mencapai 1245 pada seed 22, tetapi juga mempunyai simpangan baku tertinggi, yaitu 49,89.

Populasi 40 memulai pencarian dengan rerata individu terbaik 1136,67, lebih tinggi daripada 1103,33 pada populasi 10 dan 20, tetapi hasil akhirnya tetap lebih rendah. Perubahan ukuran populasi mengubah sampel awal dan konsumsi bilangan acak, sehingga seed yang sama tidak menjamin lintasan evolusi yang sama. Data ini tidak mendukung anggapan bahwa populasi lebih besar selalu menghasilkan solusi lebih baik.

Elitisme mempertahankan individu terbaik sehingga nilai maksimum populasi tidak menurun. Rata-rata populasi tetap dapat berubah setelah seleksi, crossover, repair, dan mutasi. Pada B40, rerata fitness populasi terakhir yang dirata-ratakan antar-seed adalah 1010,79, sedangkan rerata individu terbaik adalah 1145. Perbedaan tersebut menunjukkan bahwa kualitas elit perlu dibaca bersama kualitas anggota populasi lainnya.

3.5 Perbandingan durasi, konsistensi, dan verifikasi

Pada `single_value`, Stochastic mencatat durasi rerata terendah sebesar 0,5405 detik, disusul SA 1,4782 detik dan GA A50 2,0620 detik. Steepest Ascent memerlukan 6,0079 detik, Random Restart 30,0686 detik, dan Sideways 47,5103 detik. Random Restart memberikan rerata nilai tertinggi dan mencapai optimum pada semua seed, dengan biaya waktu lebih besar daripada Steepest Ascent.

Sideways memakai waktu jauh lebih lama daripada Steepest Ascent tanpa menambah nilai akhir pada ketiga seed. Pada `multi_value`, Steepest Ascent mencapai optimum pada seluruh seed dengan rerata 6,2349 detik, sedangkan Sideways dan Random Restart masing-masing memerlukan 24,0988 dan 22,1588 detik. Rerata durasi Stochastic, SA, dan GA pada kasus tersebut adalah 0,4349, 0,9080, dan 2,6711 detik.

Satu iterasi mempunyai biaya berbeda antaralgoritma. Steepest Ascent dan Sideways memindai seluruh neighbor valid; Random Restart mengulang pemindaian dari state awal baru. Stochastic dan SA mengambil kandidat melalui sampling, sedangkan GA membentuk serta mengevaluasi populasi. SA juga melakukan pemeriksaan keberadaan neighbor yang meningkatkan nilai objektif untuk mencatat kondisi stuck.

Perbandingan kualitas turut dipengaruhi anggaran inisialisasi. GA memilih individu terbaik dari populasi awal, sedangkan HC dan SA memulai dari satu state; Random Restart mencoba beberapa inisialisasi. Simpangan baku pada tiga seed hanya menggambarkan variasi hasil yang diamati. Dua dataset dan satu jadwal suhu SA belum cukup untuk menentukan algoritma yang unggul pada berbagai jenis muatan.

Pemeriksaan 54 hasil memastikan state awal dan akhir lolos validator, nilai objektif akhir sesuai evaluator terpilih, serta peta koordinat awal dan akhir sesuai isi state. Seluruh individu populasi awal GA juga valid. Data problem pada JSON sesuai dengan input satu truk dan multi-truk yang digunakan.

Riwayat GA menunjukkan nilai maksimum tidak menurun dan rata-rata fitness tidak melebihi maksimum. Pada SA, probabilitas berada dalam rentang [0, 1], jadwal suhu sesuai rumus linear, dan kandidat dengan $\Delta E \geq 0$ diterima. Hitungan episode stuck pada riwayat sesuai dengan metadata hasil.

BAB 4

Kesimpulan dan Saran

4.1 Kesimpulan

Pada kasus satu truk dengan 12 paket, Random Restart mencapai optimum global 1245 pada seluruh seed. Steepest Ascent, Sideways, dan Stochastic mencapai nilai tersebut pada dua dari tiga seed. SA menghasilkan 890, 1150, dan 1185, sedangkan GA A50 menghasilkan 1125, 1185, dan 1175. Stochastic mencatat durasi rerata terendah, yaitu 0,5405 detik, dengan variasi nilai akhir yang lebih besar daripada Steepest Ascent.

Pada kasus tiga truk dengan 15 paket, Steepest Ascent, Sideways, dan Random Restart mencapai optimum global 1285 pada semua seed. Stochastic mencapainya pada satu seed, sedangkan SA dan GA belum mencapai nilai tersebut. Fungsi objektif `eta_priority` mencapai jumlah kontribusi seluruh paket sebesar 362,666667 pada dua dari tiga run Steepest Ascent.

Penambahan generasi GA dari 25 menjadi 100 pada populasi 20 meningkatkan rerata nilai akhir dari 1136,67 menjadi 1191,67. Peningkatan ukuran populasi dari 10 menjadi 40 pada 50 generasi justru menurunkan rerata dari 1178,33 menjadi 1145,00 pada data ini. Hasil tersebut menunjukkan pengaruh parameter dan seed terhadap kualitas solusi.

Implementasi mendukung HC dan variasinya, SA, GA, fungsi objektif ETA, multi-truk, penyimpanan hasil, serta visualisasi 3D. Seluruh 54 hasil tersimpan konsisten dengan validator dan evaluator yang digunakan. Kesimpulan perbandingan berlaku pada dua dataset serta konfigurasi yang diuji.

4.2 Saran

SA sebaiknya menyimpan state terbaik yang pernah ditemukan secara terpisah dari state terakhir. Pada `single_value` seed 11, langkah ini akan mempertahankan rekor 995 ketika pendinginan berakhir pada 890. Variasi suhu awal dan jadwal pendinginan juga perlu diuji karena seluruh hasil SA pada parameter saat ini masih berada di bawah optimum.

Eksperimen berikutnya perlu menambah seed dan variasi muatan, termasuk kasus dengan ruang atau kapasitas yang lebih ketat. Perbandingan dengan batas waktu atau jumlah evaluasi yang sebanding, pengukuran durasi berulang, dan pencatatan lingkungan eksekusi akan membantu menilai kualitas serta biaya pencarian.

Pengujian sensitivitas dapat mencakup peluang pelewatan 30% pada pembangkitan state awal, batas sideways, jumlah restart, serta parameter crossover dan mutasi GA. Pencatatan keragaman populasi dan jumlah gen yang ditolak saat repair dapat membantu menjelaskan konfigurasi GA yang berhenti sebelum mencapai nilai maksimum.

Pembagian Tugas

Pembagian berikut mengikuti tanggung jawab anggota pada rencana kelompok. Penugasan ini mencakup program dan laporan.

NIM	Nama	Pemrograman	Eksperimen dan laporan
13524054	Wildan Abdurrahman Ghazali	Model data, representasi state, objective, validator, random state, move, JSON, Hill-Climbing, runner, integrasi, dan implementasi fitur bonus semua algoritma *hill climbing*.	Tiga run HC dan analisisnya dengan formulasi persoalan, representasi, kendala, neighborhood, metode HC, dan README.
13524066	Nathanael Gunawan	Simulated Annealing, jadwal suhu, penerimaan kandidat, penghentian, pencatatan probabilitas/stuck, serta visualisasi state awal/akhir dan pemilihan hasil, dan implementasi fitur bonus multi truk.	Tiga run SA dengan metode SA, jadwal suhu, local optima, visualisasi, perbandingan lintas algoritma, serta kesimpulan dari hasil setiap anggota.
13524132	Dika Pramudya Nugraha	Genetic Algorithm: encoding/decoding, populasi, seleksi, crossover, mutasi, perbaikan individu invalid, grafik eksperimen, skrip batch GA, dan implementasi fitur bonus *objective function*.	Delapan belas run GA dengan metode GA, konfigurasi, analisis pengaruh populasi/iterasi, dan tabel ringkasan semua algoritma.

Referensi

[1] Beyond classical search [PowerPoint slides]. (2026). Institut Teknologi Bandung. https://edunexcontentprodhot.blob.core.windows.net/edunex/2026/97453-Artificial-Intelligence-Parent-Class/461472-Beyond-Classical-Search/171817-Local-Search/file/1788061222110_2026_IF3170_Materi02-BeyondClassicialSearch?sv=2024-11-04&spr=https&st=2026-08-30T03%3A40%3A21Z&se=2028-08-29T03%3A40%3A21Z&sr=b&sp=r&sig=lsfDzNIMuPcnM9TNeoawD13JwzN6HCd7iKrvYlhKORU%3D&rsct=application%2Fpdf [diakses: 8 Oktober 2026]

[2] Tim Asisten Laboratorium Inteligensi Artifisial, "Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," Versi 1.4, 24 September 2026. [diakses: 22 September 2026]

[3] S. Russell dan P. Norvig, Artificial Intelligence: A Modern Approach, edisi ke-4. Hoboken, NJ, Amerika Serikat: Pearson, 2020. [Daring]. Tersedia: https://aima.cs.berkeley.edu/contents.html. [Diakses: 20 Oktober 2026].

[8] Matplotlib Development Team, "Matplotlib: Visualization with Python," Dokumentasi Matplotlib. [Daring]. Tersedia: https://matplotlib.org/stable/index.html. [Diakses: 20 Oktober 2026].

Lampiran

Tabel Pengerjaan Bonus

Bonus	Checklist
Implementasi alternatif objective function	✔
Implementasi seluruh algoritma hill climbing	✔
Implementasi multi truk	✔

Link Repository: https://github.com/NathanaelGun/IF3170_G20_AIAlaWND

Format Deklarasi / Surat Pernyataan Penggunaan AI

Dengan ini, kami:

Nama/NIM : 1. Wildan Abdurrahman Ghazali - 13524054
2. Nathanael Gunawan - 13524066
3. Dika Pramudya Nugraha - 13524132

Fakultas/Sekolah : STEI-K / Teknik Informatika

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Inteligensi Artifisial

Judul Tugas/Proposal : Laporan Tugas Besar 1

Menyatakan bahwa kami tidak menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

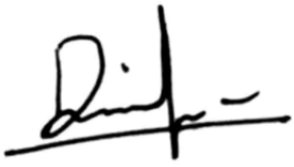
Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	3 - AI-assisted workflow

Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 - AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1 - No AI
Lainnya, Jelaskan:	Tidak	1 - No AI

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026





Wildan Abdurrahman Ghazali	Nathanael Gunawan	Dika Pramudya Nugraha
13524054	13524066	13524132